WasteWatch

Automated Illegal-Dumping Detection & Alert System — Comprehensive Technical and Operational Reference

Prepared for: Municipal and Regulatory Authority Review

Prepared by: Satyaki Solutions Pvt Ltd

Version: 1.0 — August 2026

Executive Summary

WasteWatch is a camera-based monitoring system built to detect, verify, document, and escalate instances of illegal waste dumping in public and municipal spaces. It exists to close a specific enforcement gap: municipal staff cannot watch every camera at every location at every hour, and unmonitored waste hotspots become self-reinforcing — one pile of illegally dumped waste attracts more. WasteWatch is designed to give a small enforcement team the coverage of a much larger one, without asking that team to trust an unaccountable black box.

The system's design rests on a layered philosophy: no single automated step is allowed to unilaterally produce a punitive consequence. A person lingering in a restricted zone is not a violation. A vision-language model's opinion that dumping is occurring is not, by itself, an official record. Only when object detection, behavioral timing, and AI visual verification agree — and, for the more open-ended observation mode described later in this document, only after a human reviewer explicitly confirms it — does an event become an official violation record capable of triggering escalation. This report exists to make that layered decision process fully visible to a regulatory reader, from the moment a camera captures a frame to the moment a human officer sees a confirmed case on a dashboard.

At the technical core, WasteWatch runs a real-time video analytics pipeline built on NVIDIA DeepStream. Each camera stream is decoded and batched, passed through an object detector (RF-DETR, accelerated with TensorRT) that identifies people and vehicles, and then through a multi-object tracker (ByteTrack) that follows each detected object across frames using a temporary, non-identifying tracking number. A region-of-interest filter discards anything outside operator-defined "restricted zones," and an event finite-state machine (FSM) tracks how long a given tracked object dwells inside such a zone. Only sustained or suspicious dwell behavior escalates the object toward AI verification — brief or incidental presence generates no record at all.

When escalation occurs, the system captures a still image (a close crop of the object plus a wider scene-context frame) and submits it to a Vision-Language Model — an AI system trained to interpret images and answer structured questions in natural language. Today this model is Qwen2.5-VL-7B; a planned and specified upgrade path moves this role to MiniCPM-o 4.5, a newer, more capable vision-language model, without changing the surrounding request/response contract the rest of the system depends on. In both cases, the model is asked to describe the observed action, judge whether it resembles dumping, classify the type of waste if any, and provide a confidence score and severity rating. This judgment — not the raw detection — is what actually determines whether an alert is raised. The entire AI inference step runs on infrastructure controlled by the operating authority. No camera imagery is transmitted to a third-party commercial AI service as part of this analysis.

A second, planned enhancement extends the system's coverage beyond dwell-time-triggered checks. Rather than only reacting when an object lingers, a continuous "narration" mode will maintain an ongoing AI description of each camera's scene, built on MiniCPM-o 4.5's native real-time streaming architecture (internally called Omni-Flow), which processes video in aligned one-second chunks through a stateful session rather than one-shot requests. This is intended to catch violation types a bounding-box detector structurally cannot — the act of throwing an object over a fence, or a vehicle passing with an uncovered, spilling load, neither of which necessarily produces a "person standing still for 15 seconds" pattern. Critically, this mode's outputs are deliberately routed to a separate, lower-confidence human review queue rather than being treated as confirmed violations — reflecting the genuinely lower certainty of open-ended narrative observation compared to the structured, threshold-gated verification pipeline used for the primary detection path. This document describes this enhancement as specified and planned; it is not yet deployed in production at the time of writing, and this report says so explicitly wherever it is discussed, rather than blurring planned capability with live behavior.

Once an event is confirmed — either directly through the primary AI-verification path, or after a human reviewer promotes a narration-derived observation — the system distributes notifications proportional to severity: a live operations dashboard for all confirmed events, instant messaging alerts to designated staff, automated phone-call escalation for higher-severity or repeat cases, and an optional on-site audio deterrent message. Every confirmed and under-review event carries a retained evidentiary package: a snapshot, a short video clip spanning the moments before and after the event, the AI's structured reasoning text, camera and zone identity, a timestamp, and — once reviewed — the identity of the staff member who reviewed it and their decision. This evidentiary trail is designed to support subsequent enforcement action or dispute resolution with an auditable record, not merely a machine-generated accusation.

Privacy protection is treated as a first-order design constraint rather than an afterthought. The system does not perform facial recognition and does not attempt to determine who a detected person is — it tracks "a person" or "a vehicle" using a temporary

numeric identifier that exists only while the object remains visible on camera and carries no link to any identity system. Monitoring is confined to operator-defined restricted zones; general pedestrian movement elsewhere in a camera's field of view is not evaluated, flagged, or retained. Access to stored evidence requires authenticated, role-scoped credentials, distinguishing staff who may only view a case from staff authorized to act on one.

Architecturally, the system is composed of a set of independently deployable services communicating primarily through a Kafka message bus: the DeepStream pipeline itself; a VLM inference worker exposing the trigger-analysis path over gRPC; an alerting service handling severity gating and multi-channel fan-out; an evidence service handling recording, snapshot capture, and off-site backup synchronization; a license-plate-recognition worker for vehicles; a text-to-speech worker for on-site audio; a health-monitoring and self-healing subsystem that detects and recovers from camera or service failures; and a FastAPI backend serving the operator dashboard, case-history browser, and administrative APIs, backed by a PostgreSQL database. This document's technical chapters (Part II) walk through every one of these containers, their internal modules, and the function-level call chains connecting them, at a level of detail sufficient for independent technical audit.

This report is organized in three parts. Part I is written for a policy, regulatory, or operational-oversight audience: it explains what the system does, why each safeguard exists, and where a human being remains accountable for every consequential decision, without requiring familiarity with the underlying software engineering. Part II is a full technical architecture reference, intended for an independent engineering or security reviewer, walking through every container, module, and function-level flow in the system, including the exact data schemas moved between components. Part III collects reference appendices — the complete Kafka topic and database schema listings, the configuration reference, an expanded glossary, a testing and validation summary, and a mapping between the system's design choices and generalized data-protection and oversight principles. Together, these three parts are intended to let a reader — whether a policymaker deciding whether to approve deployment, or an engineer auditing the implementation — verify for themselves that the system behaves as this document claims.

Table of Contents

Part I — Governance and Operational Narrative

1. Purpose and Scope
2. Regulatory and Policy Context
3. System Overview and Plain-Language Flow
4. Detection Stage in Detail
5. Verification Stage — AI Visual Analysis
6. Human Oversight and Review Governance
7. Alerting and Escalation Process
8. Evidence Handling and Chain of Custody
9. Privacy and Data Protection Safeguards
10. Reliability, Monitoring and Failure Handling
11. Roadmap — Planned Enhancements
12. Risk Assessment and Mitigations

Part II — Technical Architecture

13. System Architecture Overview
14. Container Inventory
15. Ingestion Layer — MediaMTX
16. DeepStream Pipeline Internals
17. Event FSM Deep Dive
18. VLM Trigger Path Deep Dive
19. MiniCPM-o 4.5 Model Internals and Omni-Flow
20. Planned Narration Path Deep Dive
21. Kafka Messaging Layer
22. Alert and Escalation Services
23. Evidence and Recording Subsystem
24. License Plate Recognition Subsystem
25. Text-to-Speech and On-Site Audio Subsystem
26. Health Monitoring and Self-Healing
27. Backend API and Database Schema
28. Frontend and Operator Dashboard
29. Security, Authentication and Access Control
30. Deployment, Infrastructure and Capacity Planning

Part III — Appendices

A. Full Kafka Topic Reference
B. Full Database Schema Reference
C. Configuration Reference
D. Expanded Glossary
E. Diagram Index
F. Document Change Log and Versioning

G. Testing and Validation Summary

H. Compliance and Governance Principles Mapping

Part I — Governing & Operational Narrative

This Part explains, for a government, regulatory, or municipal-oversight audience, what WasteWatch does, why it is built the way it is, and — most importantly — where human judgement remains in control of every consequential decision the system makes. It is written to be readable without a technical background. Part II, which follows, provides the full technical architecture for independent audit.

Chapter 1. Purpose and Scope

WasteWatch is a camera-based monitoring system designed to detect and document instances of illegal waste dumping in public and municipal areas — roadside verges, the surroundings of public waste bins, alleys, and other locations where unauthorised dumping is a recurring problem. It exists to close a well-known enforcement gap: illegal dumping is difficult to police because it typically happens briefly, without witnesses, and leaves no reliable trace back to the person responsible once the waste has been left behind.

The system's purpose is narrow and deliberately so. It is designed to:

- Continuously observe a small number of pre-designated "restricted zones" within a camera's field of view;
- Identify when a person or vehicle lingers in such a zone in a manner consistent with dumping behaviour;
- Independently verify that behaviour using a second, purpose-built artificial-intelligence check before treating it as suspicious;
- Capture and preserve evidence of the event in a form suitable for review;
- Notify designated municipal staff so that a human being can decide what, if anything, happens next.

WasteWatch does not issue fines. It does not generate legal notices. It does not communicate with, or take any automated action against, the public. Every consequence that follows a detected event — a site visit, a warning, a penalty, a referral to enforcement — is a decision made by a person, using the evidence the system provides. The system's role ends at "detect, verify, evidence, notify."

1.1 Why This Document Exists

Municipal and regulatory bodies considering, approving, or overseeing the deployment of an automated monitoring system are entitled to a complete and honest account of how it works — not a marketing summary. This document is structured in two parts for that reason:

- **Part I (this part)** explains the system's behaviour, safeguards, and governance model in plain language, for policy review, public accountability, and non-technical oversight.
- **Part II** provides the complete technical architecture — every software component, message format, and data flow — sufficient for an independent technical auditor to verify that the system does what Part I says it does, and nothing more.

> A recurring principle throughout this document: nothing described in Part I is a summary that hides complexity in Part II. Every claim made here about safeguards, human review, or data handling is directly traceable to a specific mechanism documented in Part II. Where this document says "a human must confirm this before it becomes a record," Part II shows the exact function and database field that enforces it.

1.2 Deployment Context

WasteWatch is intended for deployment by a municipal body, waste-management authority, or delegated operator, on camera infrastructure that the deploying authority owns or has lawful access to. It is not designed, marketed, or licensed for covert surveillance, private residential monitoring, or monitoring of individuals outside the specific, geographically-bounded purpose of illegal-dumping enforcement in public spaces.

1.3 What Is Explicitly Out of Scope

Not part of this system	Why this matters
Facial recognition or personal identification	The system detects "a person," never "which person" — see Chapter 9
Automated penalty issuance	All enforcement action is a human decision — see Chapter 6
General-purpose surveillance	Only pre-defined restricted zones are evaluated — see Chapter 4
Data sale or third-party sharing	All processing and storage stays within operator-controlled infrastructure — see Chapter 9

Predictive or profiling analytics on individuals	Tracking identifiers are session-scoped and discarded, not aggregated into personal profiles — see Chapter 9

Chapter 2. Regulatory and Policy Context

This chapter sets out the general regulatory principles that inform the system's design. It intentionally does not cite specific national or regional statutes, since the appropriate legal framework depends on the jurisdiction in which the system is deployed and should be confirmed by the deploying authority's own legal counsel against locally applicable law. What is provided here is a description of the underlying policy principles the system was engineered to respect, so that a reviewing body can map them onto its own applicable legal requirements.

2.1 Proportionality of Public-Space Monitoring

A foundational principle in public-space camera monitoring is proportionality: the intrusion into ordinary public life should be no greater than necessary to achieve the stated public-interest purpose. WasteWatch is designed around this principle in several concrete ways:

- **Purpose limitation.** The system evaluates only for the presence of illegal-dumping-consistent behaviour within defined restricted zones. It has no capability, and is not configured, to monitor for any other purpose.
- **Data minimisation.** Ordinary pedestrian and vehicle movement that does not meet the dwell-time and behavioural criteria described in Chapter 4 is not retained. No general video archive of public movement is created by this system's normal operation.
- **Necessity of AI verification.** The second-stage AI check exists specifically to reduce false positives — see Chapter 5 — so that fewer, more accurate events reach a human reviewer or are retained as evidence, rather than a wide net of raw motion-detection alerts.

2.2 Evidentiary Standards

Where evidence gathered by this system may be used to support an enforcement action, it should meet standards of reliability and traceability comparable to any other form of documentary evidence used by the deploying authority. The system is designed to support this through:

- Timestamped, tamper-evident evidence records (Chapter 8);
- A documented chain from raw detection through AI verification to human confirmation (Chapters 4–6);
- A retained log of who reviewed each case and what decision was made (Chapter 6).

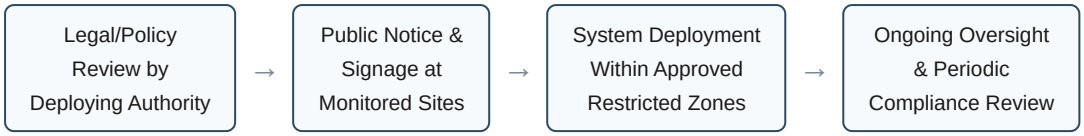
The deploying authority remains responsible for determining whether, and how, evidence produced by this system satisfies the evidentiary rules of its own jurisdiction and enforcement process (for example, whether an administrative notice, a court proceeding, or an internal disciplinary process is contemplated, and what standard of proof each requires).

2.3 Data Protection Principles

The system's data-handling design reflects commonly recognised data-protection principles — lawfulness, purpose limitation, data minimisation, storage limitation, integrity, and accountability — as elaborated in Chapter 9. A deploying authority operating under a specific data-protection statute (for example, a national personal-data-protection act, or a comparable regional framework) should conduct its own formal data-protection impact assessment using the technical detail in Part II, and may need to register the deployment, publish notices to the public, or take other steps required by that specific law. This document is a technical and operational aid to that process, not a substitute for it.

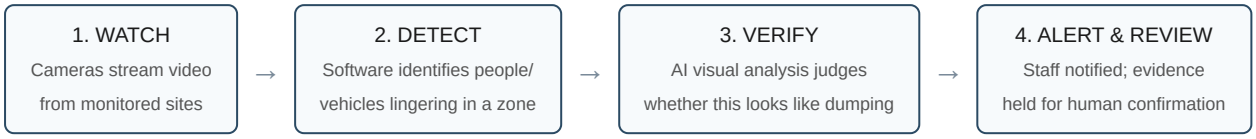
2.4 Transparency and Public Notice

Good practice for public-space camera monitoring generally includes visible signage indicating that camera monitoring is in operation, the purpose of that monitoring, and contact information for enquiries. WasteWatch does not itself provide physical signage — this is a deployment-site responsibility for the operating authority — but the system's design (narrow purpose, no facial recognition, no personal profiling) is intended to make such a notice both accurate and complete when issued.


Legal/Policy Review by Deploying Authority
Public Notice & Signage at Monitored Sites
System Deployment Within Approved Restricted Zones
Ongoing Oversight & Periodic Compliance Review

Chapter 3. System Overview and Plain-Language Flow

At the highest level, WasteWatch operates as a four-stage pipeline. Every event the system produces passes through all four stages in order; none can be skipped, and each stage's output is the necessary input to the next.



3.1 Stage 1 — Watch

Cameras already installed, or newly installed for this purpose, at monitored locations stream continuous video into the system. This is functionally no different from any conventional CCTV deployment at the point of image capture — the system does not require special cameras, only a stable video feed (via standard network video protocols) and a suitable vantage point over the area of concern.

3.2 Stage 2 — Detect

Software analyses the incoming video in real time to identify people and vehicles, and to determine whether any of them are within a "restricted zone" — a boundary drawn by the municipal operator onto that camera's view, corresponding to the specific area (for example, the immediate vicinity of a public bin) where dumping monitoring is intended. Detection alone does not create any record; it only determines whether closer attention (Stage 3) is warranted, based on how long a person or vehicle remains in the zone. This is detailed fully in Chapter 4.

3.3 Stage 3 — Verify

When Stage 2's criteria are met, the system does not assume a violation has occurred. Instead, it captures the relevant image(s) and submits them to a dedicated AI visual-analysis step, which independently assesses whether the observed behaviour is consistent with dumping, what type of waste appears to be involved, and how confident that assessment is. This is detailed fully in Chapter 5.

3.4 Stage 4 — Alert and Review

Only once Stage 3 has produced a positive assessment does the system generate a notification to designated staff and create a reviewable evidence record. A human being — not the software — makes the final determination as to whether the event is genuine and what follow-up, if any, is warranted. This is detailed fully in Chapter 6.

3.5 What Happens When a Stage Says "No"

At every stage, the default outcome is that nothing is recorded. A person walking past a restricted zone without lingering never reaches Stage 3. A person lingering, but whom the AI verification step assesses as not dumping (for example, someone waiting, or attending to a bicycle), never reaches Stage 4. Only events that pass every stage's check become part of the record that a human being sees. This design deliberately biases the system toward under-reporting rather than over-reporting — see Chapter 12 for the corresponding discussion of false negatives as an accepted trade-off against false positives.

Chapter 4. Detection Stage in Detail

The detection stage is the system's first line of observation. Its job is narrow: identify people and vehicles, determine whether they are within a restricted zone, and measure how long they remain there. It does not, by itself, make any judgement about intent or wrongdoing.

4.1 What the Software Identifies

The detection software is trained to recognise general categories of object relevant to dumping enforcement — principally people, and vehicles such as cars, motorcycles, and trucks (vehicles matter because a common form of illegal dumping involves a vehicle stopping briefly to unload material). It does not attempt to identify a specific named individual, read identifying features, or perform any biometric matching. See Chapter 9 for the full privacy treatment of this point.

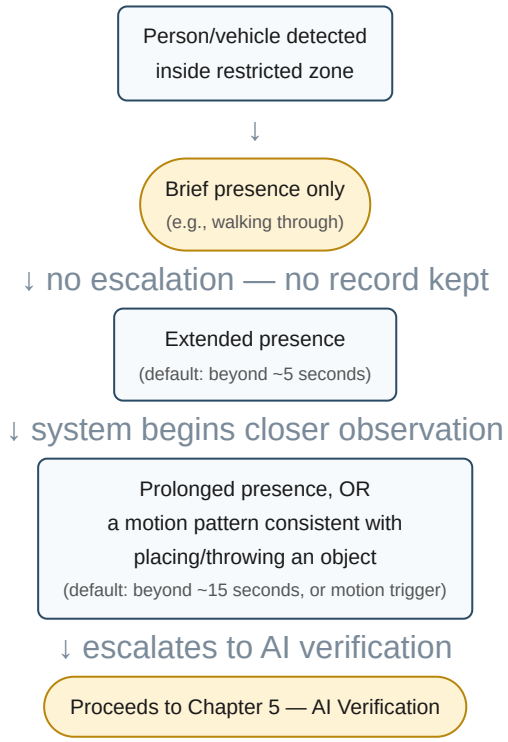
4.2 Restricted Zones

A restricted zone is a boundary the municipal operator draws directly onto a given camera's view, using a configuration tool, to mark the specific area where dumping monitoring should apply — for instance, a five-metre radius around a public bin, or a stretch of roadside verge with a documented dumping history. Activity outside a defined restricted zone is not evaluated for violations under this system; a person walking on a public footpath adjacent to, but outside, a marked zone is not subject to escalation regardless of how long they remain in view.

Restricted zones are configured per camera and per site, allowing the system to be tailored to the specific geography and known problem areas of each deployment location, rather than applying a single blanket rule everywhere.

4.3 The Escalation Ladder

Once a person or vehicle is detected within a restricted zone, the system does not react instantly. It observes how long they remain and escalates its level of attention in stages, only reaching the point of AI verification (Chapter 5) if the observed behaviour continues beyond a threshold that ordinary, innocent use of the area would not typically involve.



These time thresholds are configurable per camera by the municipal operator, in recognition of the fact that "normal" dwell time varies by location — a restricted zone adjacent to a bus stop or a loading bay would reasonably need a longer threshold than an isolated alley with no legitimate reason for anyone to linger.

4.4 Tracking Without Identity

While a person or vehicle remains visible within the camera's view, the system assigns them a temporary tracking number so that its escalation ladder (above) can measure continuous dwell time for that specific individual, rather than resetting every time they move slightly within frame. This tracking number:

- Is generated fresh each time a person enters the camera's view;
- Is discarded once they leave the camera's view;
- Is never linked to a name, a government identifier, a facial signature, or any other means of personal identification;
- Cannot be used to recognise the same individual on a later, separate visit to the same or a different camera.

> This is a structural, not a policy, limitation — the system's detection and tracking components have no facial-recognition or identity-matching function built into them at all. There is no capability to disable in order to achieve this safeguard; the capability to identify individuals simply does not exist in the software.

4.5 Multi-Camera Operation

Where multiple cameras are deployed across a site or municipality, each operates its own independent detection and tracking process for its own restricted zones. The system supports a substantial number of simultaneous camera feeds (in current deployments, on the order of ten to fifty), processed in a shared but resource-managed fashion so that no single camera's activity starves attention from another.

Chapter 5. Verification Stage — AI Visual Analysis

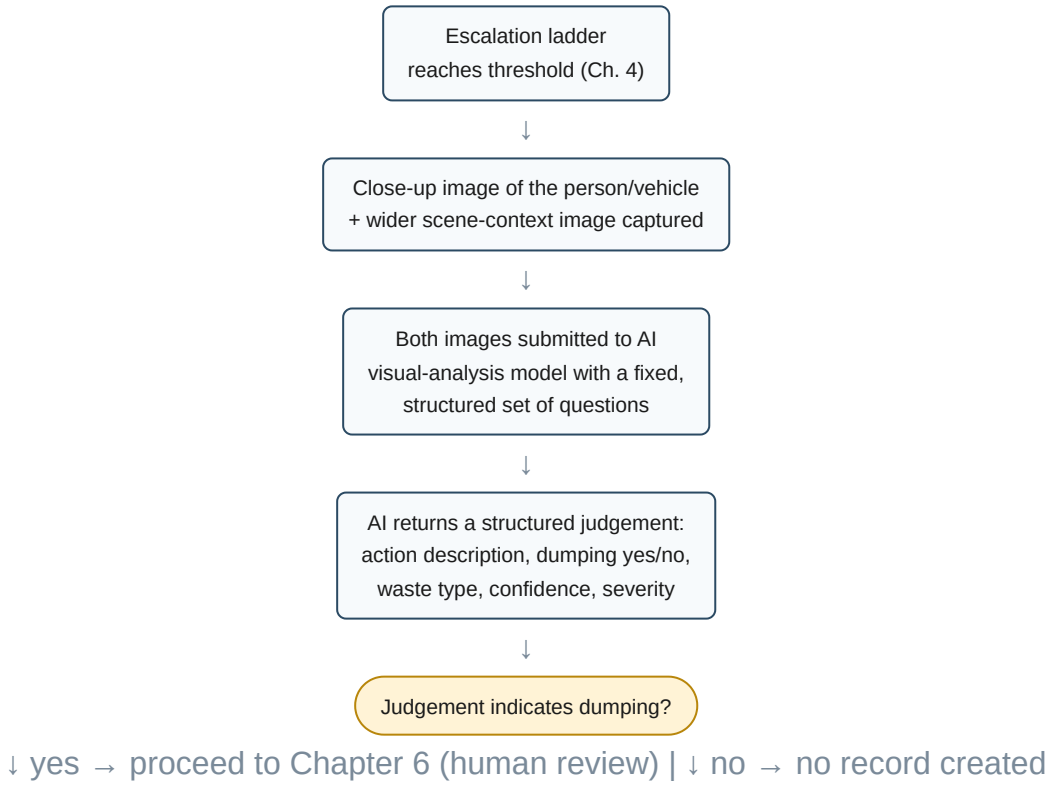
Reaching the top of the escalation ladder in Chapter 4 does not create a record of wrongdoing. Prolonged presence in a restricted zone is, at that point, still only a behavioural signal — it is equally consistent with someone waiting, resting, tying a shoelace, or attending to a vehicle, as it is with dumping. The purpose of the verification stage is to apply an independent, visually-grounded check before anything is treated as suspicious.

5.1 Why a Second, Independent Check

Relying on dwell time and motion alone would produce a high rate of false alarms — most systems that alert purely on "someone stood still for N seconds" generate far more noise than actionable signal, which in practice trains staff to ignore alerts altogether (a well-documented failure mode in security and monitoring systems generally, sometimes called "alert fatigue"). WasteWatch addresses this directly by inserting a qualitatively different kind of check — one that looks at the actual visual content of the moment, rather than only its timing — before anything reaches a person's attention.

5.2 What the AI Verification Step Actually Does

When the escalation ladder reaches its threshold, the system captures a still image of the triggering moment (a close, cropped view of the person/vehicle in question) together with a wider contextual image of the same scene, and submits both to a Vision-Language Model — an artificial-intelligence system trained specifically to interpret the content of images and describe it in structured, natural language.



5.3 The Structured Question the AI Is Asked

The model is not asked an open-ended "what do you see" question. It is given a fixed, structured prompt equivalent in substance to:

> "Describe what this person or vehicle is doing in the image. Does this look like waste is being dumped or discarded here? If so, what type of waste appears to be involved? State your confidence in this assessment, and how severe this appears to be."

The model's response follows a fixed structure — a plain-language description of the action, a yes/no judgement on whether dumping appears to be occurring, an estimated waste category (for example, general refuse, construction debris, or similar), a numeric confidence score, and a severity rating. This structure is what allows the system to apply consistent downstream rules (Chapter 6, Chapter 7) rather than trying to interpret free-form text.

5.4 Confidence and Severity as Explicit, Visible Values

The AI's confidence score and severity rating are not hidden internal numbers — they are retained as part of the event record and are visible to the human reviewer described in Chapter 6, alongside the AI's own stated reasoning. This ensures a reviewer is not being asked to simply trust a system's yes/no output blind; they can see why the system reached that judgement and how sure it says it is.

5.5 Where the AI Model Runs

The Vision-Language Model used for this verification step is hosted entirely on computing infrastructure controlled by the operating authority — either on-premises servers or a private cloud instance under the operator's own administrative control. No image is transmitted to any external, third-party commercial AI service as part of this process. This is a deliberate architectural choice, not an incidental one: it keeps sensitive imagery of public spaces within the operator's own governance boundary at all times.

5.6 Model Selection and Evolution

The specific AI model performing this analysis is expected to evolve over the system's operational life as better-performing, more efficient, or more capable models become available — in the same way that antivirus signature databases or spam filters are periodically updated without changing the surrounding process they serve. The current production model, and a planned successor model under evaluation, are documented fully in Part II, Chapters 18 and 19. What does not change with a model update is the structure described in this chapter: a fixed question, a structured answer, and a human reviewer downstream who sees the AI's stated reasoning rather than a bare verdict.

5.7 Handling AI Uncertainty and Failure

The system is designed to fail toward inaction, not toward false alarms, when the AI verification step is uncertain or unavailable:

- If the AI's response cannot be understood in the expected structured format, the system automatically retries once with a more explicit prompt;
- If it still cannot obtain a valid structured response, the event is recorded with the lowest confidence and severity levels rather than being escalated as if it were a confirmed high-severity case;
- If the AI service itself becomes unavailable for a sustained period, the system detects this automatically and temporarily suspends verification attempts rather than repeatedly failing against a known-broken service, while continuing to log that verification coverage has been interrupted so operators are aware.

Chapter 6. Human Oversight and Review Governance

This chapter describes what is, from a governance perspective, the single most important design feature of the system: no automated finding — regardless of the AI's confidence score — becomes an actionable, official record without a documented point of human decision-making.

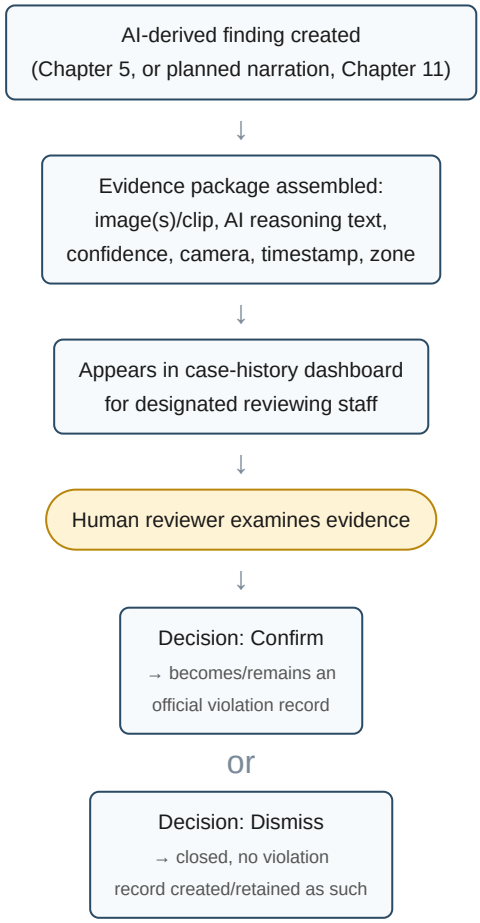
6.1 Two Tiers of Confidence, Two Levels of Human Involvement

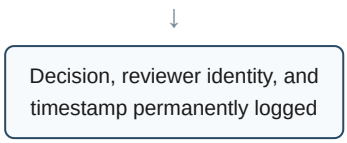
The system distinguishes between two categories of AI-derived finding, and treats them with different degrees of human involvement calibrated to how structured and reliable each category is:

Finding type	How it arises	Human involvement
Structured, threshold-verified event	Escalation ladder (Ch. 4) + AI verification (Ch. 5), both against a fixed, repeatable process	Becomes a recorded event with full evidence immediately; a human reviewer can inspect, confirm, or dispute it in the case-history dashboard, and their decision is logged
Open-ended AI observation *(planned enhancement, see Ch. 11)*	Continuous scene narration flags something unusual, independent of the escalation ladder	Held in a dedicated review queue in a "pending" state; does **not** become an official violation record unless and until a human reviewer explicitly confirms it

> The second category exists specifically because open-ended AI observation is inherently less structured and less reliable than a fixed, repeatable verification process. Rather than treat every AI utterance as equally authoritative, the system's design deliberately routes less-structured findings through an additional, mandatory human gate before they can have any operational consequence.

6.2 The Review Workflow





6.3 Accountability of the Human Decision Itself

The system does not merely allow a human to review a finding — it records that review as its own accountable event. Every confirm or dismiss decision is logged with the identity of the staff member who made it and the time it was made. This creates a two-sided accountability record: the AI's reasoning is visible to the reviewer, and the reviewer's decision is visible to any subsequent audit. Neither the machine nor the human decision-maker operates in an unrecorded, unaccountable manner.

6.4 Role-Based Access to Review Functions

Not every system user can confirm or dismiss a finding. The system supports role-based access control (detailed fully in Part II, Chapter 29), distinguishing at minimum between staff who can view evidence and staff who are authorised to make a confirm/dismiss decision. This ensures that the power to convert an AI finding into an official record — or to close one out — is limited to designated, accountable personnel, consistent with normal municipal delegation-of-authority practice.

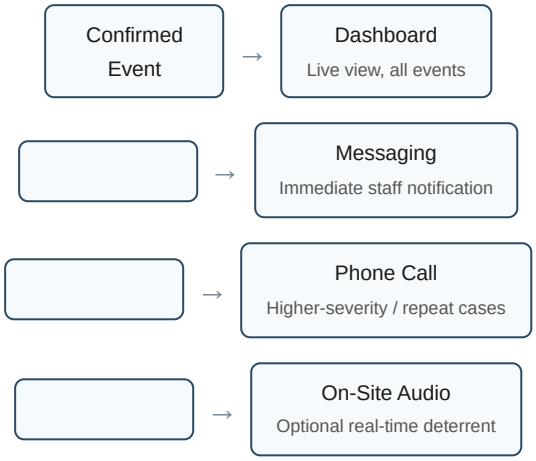
6.5 What "Confirmed" Enables, and What It Does Not

Confirming a finding creates an official violation record, with its full evidence package, that becomes visible in the alerting and reporting functions described in Chapter 7. It does not, by itself, issue any penalty, fine, or notice — those remain separate actions taken by the deploying authority's own enforcement process, using the confirmed record as supporting evidence.

Chapter 7. Alerting and Escalation Process

Once an event is either directly verified (Chapter 5) or confirmed by human review (Chapter 6), the system's role shifts from detection to notification: making sure the right staff know about it, promptly, through the right channel for its severity.

7.1 Notification Channels



Channel	Typical use	Who receives it
Operations dashboard	Every confirmed event, viewable live and historically	Monitoring/operations staff
Messaging notification	Prompt notice of a new event as it happens	Designated duty officers
Automated phone call	Higher-severity events, or repeated activity by the same session-tracked individual	On-call enforcement/response staff
On-site audio announcement	Optional real-time deterrent message played at the camera location itself	The person present at the site, as a deterrent, not a notification to staff

7.2 Severity-Based Routing

Not every confirmed event is routed identically. The AI verification step's severity rating (Chapter 5) determines how urgently a case is escalated — a low-severity, low-confidence event may appear only on the dashboard for routine review, while a high-severity event, or a repeat occurrence at the same location by the same tracked session, can trigger the phone-call escalation channel to ensure timely human attention.

7.3 Repeat-Activity Escalation

Where the same tracked individual (identified only by the temporary, session-scoped tracking number described in Chapter 4 — never by personal identity) is observed triggering multiple events within a single continuous visit, the system can raise the severity of its response, recognising that persistent activity at one location warrants faster attention than an isolated incident.

7.4 The On-Site Deterrent Message

Where enabled, the system can generate a spoken announcement, played through a speaker at the monitored location, addressed to the person currently present — for example, asking them to stop and remove the material. This is a real-time deterrent measure, generated from the AI verification step's own description of the scene, intended to prevent the dumping from being completed rather than only to document it after the fact. It is a site-level, optional feature configured by the operator per location.

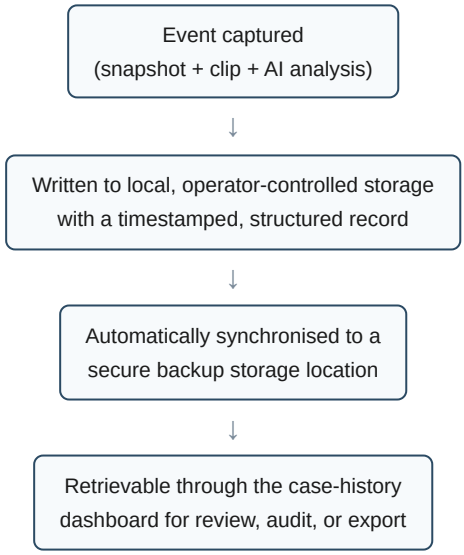
Chapter 8. Evidence Handling and Chain of Custody

Where an event may support an enforcement or disciplinary action, the reliability of its supporting evidence matters as much as the detection itself. This chapter describes what is captured, how it is protected against loss or tampering, and how it can be traced back through the process that produced it.

8.1 What Is Captured for Every Event

Evidence item	Purpose
Snapshot image at the moment of detection	Primary visual evidence of the triggering moment
Short video clip spanning before and after the event	Context — shows the lead-up and aftermath, not just an isolated frame
The AI's structured analysis and stated reasoning	Explains why the system flagged this event, supporting review (Chapter 6)
Camera identity, timestamp, and restricted-zone identifier	Establishes exactly where and when the event occurred
Review outcome and reviewing staff identity, once reviewed	Establishes the human accountability trail described in Chapter 6

8.2 Evidence Lifecycle



8.3 Integrity and Immutability

Once written, an evidence record is not designed to be silently altered — subsequent actions (such as a human review decision) are appended as additional, separately timestamped entries associated with the original record, rather than by rewriting the original evidence itself. This preserves a traceable history of the record from its creation through to its final reviewed status, supporting its use as a reliable account of what the system observed and when.

8.4 Backup and Redundancy

Evidence is not held in only one location. In addition to the primary on-premises or operator-controlled storage, records are automatically synchronised to a secure, access-controlled cloud storage location, reducing the risk that a single hardware failure or site incident results in the loss of evidence relevant to an ongoing or prospective enforcement matter.

8.5 Retrieval and Export

Authorised staff can retrieve evidence for a given event, camera, date range, or review status through the case-history dashboard described in Chapter 6 and detailed technically in Part II, Chapter 27–28. This supports both routine operational review and the preparation of a specific case file where an event proceeds toward a formal enforcement action.

8.6 Retention

The length of time evidence is retained is a policy decision for the deploying authority to set, balancing the operational need to support enforcement or appeal processes against data-minimisation principles (Chapter 9). The system's storage architecture supports a configurable retention policy; the specific duration is not fixed by the software itself and should be determined by the operator in line with applicable local record-retention rules.

Chapter 9. Privacy and Data Protection Safeguards

This chapter consolidates, in one place, every structural safeguard the system applies to protect individual privacy while still achieving its enforcement purpose. Several of these points have been introduced in earlier chapters; they are restated here together because privacy protection in this system is not a single feature but a set of interlocking design choices.

> **No facial recognition.** The detection stage (Chapter 4) identifies general categories — "a person," "a vehicle" — and has no built-in capability to extract, store, or match facial features against any identity database. This is a structural absence, not a disabled option.

> **Temporary, session-scoped tracking only.** The tracking identifier assigned to a detected person exists only for the duration they remain visible in a single camera's view and is discarded afterward. It cannot be used to recognise the same person across a later, separate visit, across different cameras, or to build any profile of an individual's movements over time.

> **Zone-limited evaluation.** Only pre-defined restricted zones (Chapter 4) are evaluated for violation criteria. Movement elsewhere in a camera's field of view is not subject to the escalation ladder, AI verification, or record creation.

> **Data minimisation by design.** The large majority of observed activity — brief presence, presence outside a restricted zone — creates no record at all. The system is architected to generate evidence only for the narrow set of events that pass every stage of Chapters 4–6, not to build a comprehensive video archive of public activity.

> **Self-hosted AI processing.** The AI verification step (Chapter 5) runs entirely on infrastructure controlled by the operating authority. No image or video derived from the monitored cameras is transmitted to a third-party commercial AI provider as part of this system's normal operation.

> **Access-controlled evidence.** Retrieval of stored evidence (Chapter 8) requires authenticated access, with role-based permissions distinguishing staff who may view evidence from staff who may act on it (Chapter 6.4).

> **Human-in-the-loop for less-certain findings.** As described in Chapter 6, the least structured category of AI output is deliberately prevented from becoming an official record without explicit human confirmation.

9.1 A Note on Third-Party Rights and Bystanders

Because the system monitors public or semi-public space, individuals other than the person triggering an event may incidentally appear in captured imagery (for example, someone walking through the background of a scene-context image). The same minimisation principles apply to them as to the primary subject: no facial recognition, no identity linkage, and no retention beyond what is captured as part of a qualifying event's evidence package.

9.2 Data Subject Considerations

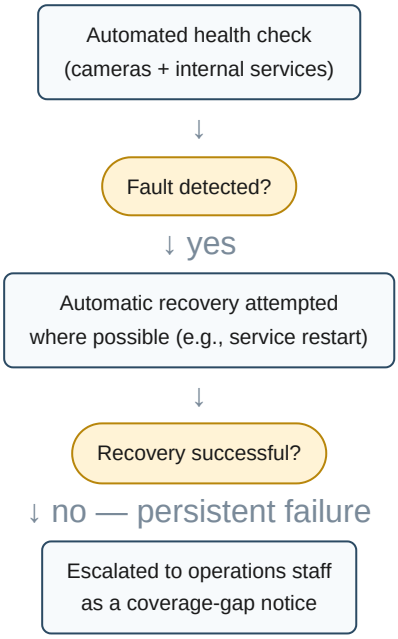
Where the deploying authority's applicable law grants individuals rights over data concerning them (for example, a right to be informed that monitoring is in operation, or a right to request access to or erasure of a specific record), the system's structured, timestamped, camera-and-zone-indexed evidence records (Chapter 8) are designed to make it operationally feasible to locate and act on a specific request, rather than requiring a manual search through an undifferentiated video archive.

Chapter 10. Reliability, Monitoring and Failure Handling

A monitoring system that silently stops working is worse than no system at all, because it creates false confidence that an area is being watched when it is not. This chapter describes how WasteWatch is designed to make its own failures visible rather than silent.

10.1 Continuous Health Monitoring

Every camera feed and every internal processing component is continuously monitored for correct operation. If a camera loses signal, goes offline, or a processing component stops responding, this is detected automatically rather than relying on a staff member to notice a blank dashboard tile.



10.2 Automatic Recovery

Where a fault is of a kind the system can resolve on its own — for example, a transient network interruption to a camera stream, or an internal service that has stopped responding but can be safely restarted — this is attempted automatically, without requiring a staff member to intervene for routine, recoverable issues.

10.3 Escalation of Persistent Failures

Where automatic recovery does not resolve the issue, the system does not retry indefinitely in silence. It surfaces the failure as an explicit notice to operations staff, distinguishing "this camera/zone currently has no monitoring coverage" from "this camera/zone is being monitored and found nothing." This distinction matters for any subsequent question about why an event during a coverage gap was not detected.

10.4 Capacity and Load Considerations

As additional cameras or AI-verification load are added to a deployment, the system's underlying processing capacity (documented technically in Part II, Chapter 30) must be sized accordingly. Reliability monitoring includes awareness of resource strain, so that a deployment approaching its processing capacity is identified before it begins silently dropping frames or delaying verification, rather than after.

Chapter 11. Roadmap — Planned Enhancements

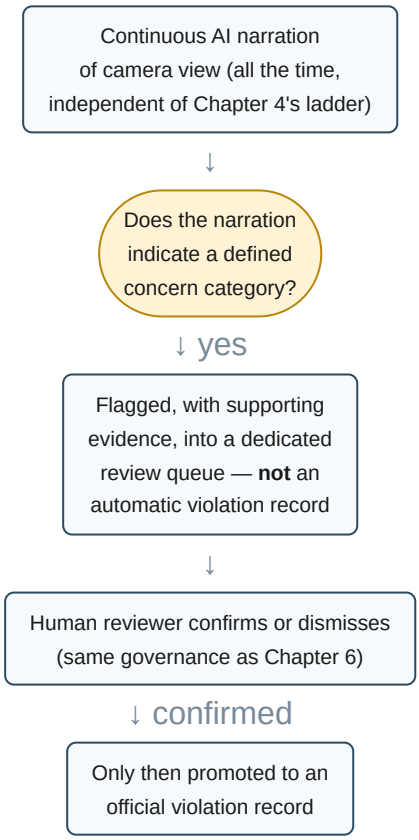
This chapter describes an enhancement currently in design at the time of writing, not yet in operational use. It is included here so that a reviewing body considering this system understands not only its current capability but the direction of its planned evolution, and can assess the safeguards that will apply to the new capability before it is deployed.

11.1 The Gap This Enhancement Addresses

The detection-and-escalation process described in Chapter 4 is built around dwell time within a defined zone — it is well suited to catching a person who lingers while dumping material. It is structurally less suited to catching an act that does not require lingering at all: for example, a person briefly throwing an item over a fence into a restricted area without ever entering it themselves, or a vehicle passing through with an uncovered or overflowing load, visible only for the moments it takes to pass by.

11.2 Continuous Scene Narration

The planned enhancement adds a second, independent AI process that does not wait for the escalation ladder at all. Instead, it maintains an ongoing, real-time description of activity within each camera's view, continuously, and is specifically asked to watch for a small number of defined behaviour categories that a dwell-time-based system would otherwise miss, together with an open-ended catch-all category for anything else unusual.



11.3 Why This Is Held to a Stricter Standard Than Chapter 4–5's Process

Continuous, open-ended narration is inherently less structured than the fixed, repeatable question-and-answer verification described in Chapter 5. Rather than treat an open-ended observation with the same weight as a threshold-triggered, structured verification, the system's design routes every narration-derived flag through the dedicated, mandatory human-review queue described in Chapter 6.1 — no exceptions, regardless of the AI's stated confidence on any individual flag. This is a deliberate governance decision, not a placeholder for a future removal of human review.

11.4 Technical Basis

This enhancement is intended to use a newer generation of AI visual-analysis model capable of processing video as a continuous stream — internally, in one-second segments — rather than only single still images, allowing it to observe motions and actions (like a throwing motion) that a single-frame analysis cannot capture. The full technical description of this mechanism, including how it interacts with the underlying video-processing pipeline, is provided in Part II, Chapters 19–20, for independent technical review.

11.5 Deployment Status

At the time of writing, this capability exists as an approved design and an implementation plan, with the review-queue governance mechanism (Chapter 6.1) built and tested against simulated conditions. It has not yet been connected to a live, deployed AI model or live camera feeds in production. This document will be updated to reflect operational status once that occurs.

Chapter 12. Risk Assessment and Mitigations

No automated system is risk-free. This chapter sets out, candidly, the principal risks associated with WasteWatch's operation and the specific design mitigation applied to each — consistent with the principle stated in Chapter 1 that this document should provide a complete and honest account, not a marketing summary.

Risk	Description	Mitigation
False positives	The system incorrectly flags innocent behaviour as dumping	Two-stage verification (Chapters 4–5) reduces raw motion-based false alarms; human review (Chapter 6) provides a final check before any record becomes an accepted violation; confidence and severity scores are visible to reviewers to support informed judgement
False negatives	The system fails to detect an actual dumping event	Acknowledged as an accepted trade-off of the system's conservative, dwell-time-and-verification design (Chapter 3.5); the planned continuous-narration enhancement (Chapter 11) is intended to reduce this specific gap for non-lingering acts, itself subject to human review before any consequence follows
AI model drift or degraded accuracy over time	A deployed AI model's performance may change as conditions (lighting, camera angle, seasonal foliage, etc.) diverge from its training conditions	Human review (Chapter 6) remains in the loop regardless of model performance; confidence scores are retained per event, allowing operators to monitor for a declining trend and prompt model review/retraining; model updates are treated as a planned, reviewed change (Chapter 5.6), not a silent one
Privacy risk from public-space monitoring	Any camera-based monitoring of public space carries an inherent privacy cost to the people observed	Comprehensive structural safeguards detailed in Chapter 9 — no facial recognition, session-scoped tracking only, zone-limited evaluation, data minimisation by design, self-hosted processing, access-controlled evidence
Operational/technical failure	A camera, network link, or processing component fails, creating a monitoring gap	Continuous automated health monitoring with auto-recovery and explicit escalation of unresolved faults (Chapter 10), so gaps are visible to operators rather than silent
Misuse of evidence or review access	A staff member with system access could misuse evidence or make an unjustified confirm/dismiss decision	Role-based access control (Chapter 6.4), and a permanent, attributable log of every review decision (Chapter 6.3) supporting after-the-fact audit and accountability
Over-reliance on AI judgement	Staff may be tempted to treat AI confidence scores as determinative rather than advisory	Governance and training should explicitly reinforce that the AI's output is a recommendation (Chapter 6), not a verdict; the system displays AI reasoning text, not just a bare score, to support genuine human judgement rather than rubber-stamping

12.1 Residual Risk Statement

The mitigations above reduce, but do not eliminate, each listed risk. A deploying authority should treat this table as an input to its own formal risk assessment and governance process, adjusting operational procedures (staff training, review workload capacity, escalation thresholds per Chapter 4.3, and audit frequency) to the specific conditions and risk tolerance of its own deployment.

End of Part I. Part II provides the complete technical architecture underlying every mechanism described above, for independent verification.

Part II — Technical Architecture

Part I described WasteWatch in operational and governance terms: what it observes, how it decides something may be a violation, and how a human remains in control of every consequential outcome. Part II now opens the system up at the engineering level — every container, every pipeline stage, every message that crosses the system, and every function call in the two paths that matter most: the trigger-based violation analysis path that is live today, and the always-on narration path that is designed and planned but not yet deployed.

This part assumes a technical reader: a systems engineer, an auditor with a software background, or an integration partner who needs to know precisely how data moves, where it is stored, what guarantees exist at each hop, and what happens when a component fails. Where a capability described here is not yet live — principally the continuous narration path and the MiniCPM-o 4.5 model swap — this is stated explicitly at the point it is introduced, and such sections are marked with a dashed, green "planned" treatment in their diagrams.

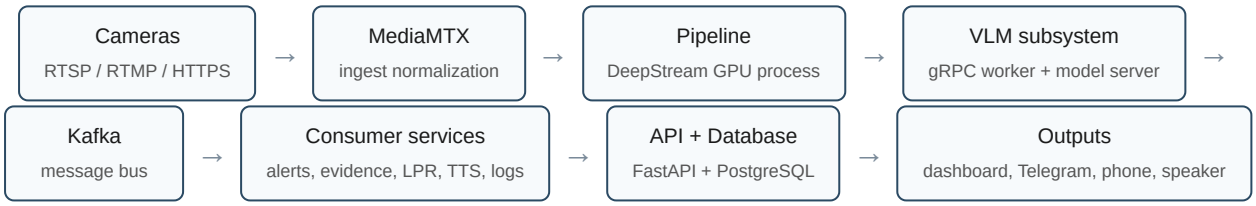
Chapter 13. System Architecture Overview

WasteWatch is built as a single-server, GPU-centric pipeline with a message bus fanning work out to a set of independent services. The design deliberately avoids splitting the GPU-bound detection and inference work across multiple processes or machines — everything that needs the GPU (object detection, AI verification, and, in the planned enhancement, continuous narration) runs inside one long-lived pipeline process and one model-serving process, both on the same physical server. Everything that does not need the GPU — alert dispatch, evidence storage, plate recognition post-processing, text-to-speech, logging, health monitoring, and the operator-facing API — runs as separate, independently restartable services connected only by asynchronous messages.

This split matters for two practical reasons. First, GPU memory is a scarce, non-shareable resource in the way ordinary CPU memory is not — running detection, tracking, and vision-language inference inside a single coordinated process lets the system reason explicitly about how much VRAM each stage consumes and avoid oversubscription. Second, decoupling everything else onto a message bus means an operator can restart the alerting service, upgrade the plate-recognition model, or take the reporting service offline for maintenance without interrupting camera ingestion or detection — the pipeline keeps running and keeps queuing events regardless of what is happening downstream.

13.1 The big picture

At the highest level, data flows in one direction — from cameras, through detection and verification, into a message bus, and out to the services that act on it — with a narrow set of feedback paths (configuration updates, camera health signals) flowing back toward the pipeline. The diagram below lays out every major hop.



Two details in this chain are easy to miss and worth stating up front. The arrow from Pipeline to VLM subsystem is not "always on" — it fires only when the tracking and zone logic inside the pipeline decides a specific tracked object has crossed a dwell-time threshold (Chapter 17 covers this in full). And the VLM subsystem's output does not go directly to the outputs on the right; it always passes through Kafka first, alongside every other signal in the system, so that every downstream consumer sees a uniform stream of typed messages rather than a web of direct point-to-point calls.

13.2 Design principles behind the architecture

13.2.1 One GPU process for GPU-bound work

The pipeline container is the only process that touches raw video frames and runs CUDA-based inference for detection and tracking. This avoids the overhead and complexity of shuttling raw or lightly-compressed frames between processes just to run a model on them. Only downstream, lightweight artifacts — JPEG-encoded crops, structured metadata, short video clips — leave the pipeline process.

13.2.2 Message bus as the only integration point downstream of detection

Every service other than the pipeline and the VLM subsystem communicates exclusively through Kafka topics (Chapter 21 gives the full topic reference). No consumer service calls another consumer service directly. This means the set of downstream services can grow — a new report generator, a new integration with an external ticketing system — without any change to the pipeline or to any existing consumer; a new service simply subscribes to the topics it needs.

13.2.3 Fire-and-forget on the hot path, durable delivery everywhere else

The one exception to pure message-passing is the trigger-based VLM call, which uses a direct gRPC request from the pipeline to the VLM worker rather than a Kafka round-trip. This is a deliberate latency optimization (detailed in Chapter 18) — the pipeline needs the crop image while it is still in memory, and adding a Kafka producer-then-consumer hop for the request side would add avoidable

delay. The gRPC call is issued fire-and-forget from a background thread pool so it can never stall the video-processing thread, and its result is published back onto Kafka exactly like every other message in the system, so downstream consumers see a consistent interface regardless of how the analysis was triggered.

13.2.4 Human decision points are architectural, not incidental

As established in Part I, the system is designed so that lower-confidence AI output (the planned narration path) is structurally routed to a separate table and a separate review workflow rather than being merged into the same code path as a confirmed violation. This is not a policy note bolted onto the software after the fact — it appears in this document as a genuine architectural decision: a different database table, a different Kafka topic, and a different set of API endpoints exist specifically so that narration-derived observations cannot silently become violation records without a human action in between (Chapter 20).

Chapter 14. Container Inventory

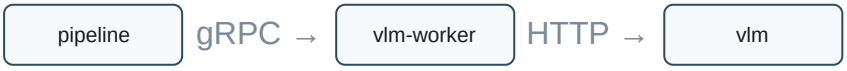
WasteWatch is deployed as a set of Docker containers orchestrated by a single Docker Compose configuration. This chapter catalogs every container, its role, and how it fits into the flow described in Chapter 13.

Container	Category	Role
mediamtx	Ingestion	Normalizes camera input protocols to RTSP; serves WebRTC to the frontend
pipeline	GPU processing	DeepStream-based detection, tracking, ROI/FSM logic, evidence capture, VLM trigger
vlm	GPU processing	Model server hosting the vision-language model (vLLM today; vllm-omni planned)
vlm-worker	GPU-adjacent service	gRPC servicer wrapping the model server, builds prompts, parses responses
kafka	Infrastructure	Message bus — every event, request, and result in the system passes through here
postgres	Infrastructure	Primary relational data store — cameras, violation events, evidence records, alerts
redis	Infrastructure	Cache and session state
qdrant	Infrastructure	Vector database backing structured log search
keycloak	Infrastructure	Identity provider — authentication and role-based access control
api	Application service	FastAPI backend — REST and WebSocket endpoints, dashboard data, camera/ROI configuration
alert-svc	Consumer service	Consumes verified violation results, applies throttling, fans out to notification channels
sip-svc	Consumer service	Places automated phone-call escalations for high-severity or repeat events
evidence-svc	Consumer service	Persists snapshots and metadata, manages pre/post-buffer video clip recording, syncs to cloud storage
lpr-worker	Consumer service	Vehicle license-plate recognition on flagged vehicle crops
tts-worker / tts	Consumer service	Generates spoken-audio deterrent messages for on-site speakers
log-worker	Consumer service	Ingests structured logs into the searchable log store
health-monitor / healer	Operations service	Watches camera and service health; automatically restarts failed components
reports	Operations service	Scheduled aggregate reporting (e.g., periodic summaries by camera/zone)

14.1 Why this grouping

The table above groups containers into five categories on purpose. Ingestion and GPU processing containers are the only two that touch live video. Infrastructure containers are commodity, off-the-shelf components (Kafka, PostgreSQL, Redis, Qdrant, Keycloak) chosen because they are mature, widely audited pieces of open-source software rather than anything custom-built for this system — this matters for a reviewing authority, because the trust surface for "is this database secure" or "is this message bus reliable" rests on well-known, independently reviewed software rather than a bespoke implementation. Application, consumer, and operations services are where WasteWatch-specific logic lives.

14.2 Container-to-container communication summary



This is the only synchronous, direct service-to-service call chain in the system. Every other inter-container link below is asynchronous, via Kafka.

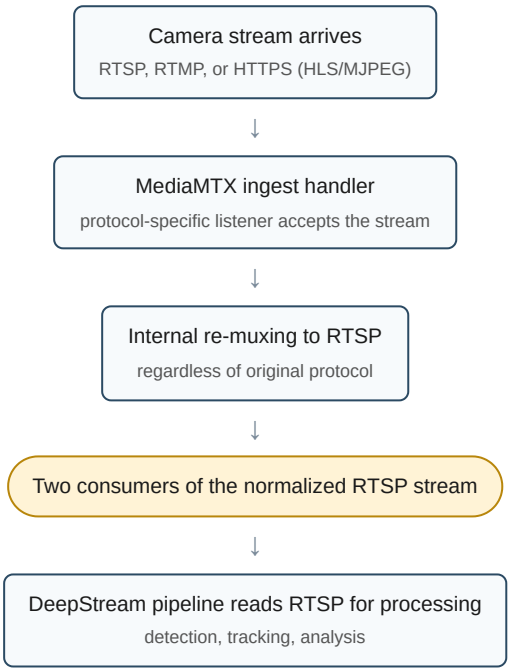


alert-svc / evidence-svc / lpr-worker / tts-worker / log-worker / api

Chapter 15. Ingestion Layer — MediaMTX

Every camera feed enters WasteWatch through MediaMTX, a media-relay server that accepts multiple ingest protocols and re-serves them in the formats each downstream consumer needs. MediaMTX exists as a dedicated layer, rather than having the DeepStream pipeline connect to cameras directly, for a simple operational reason: real-world camera deployments are heterogeneous. Some cameras only speak RTSP; some only expose RTMP; some are accessible only via an HTTPS-based HLS or MJPEG stream. Rather than building protocol-handling logic into the detection pipeline itself, MediaMTX absorbs that variability once, at the edge, and always presents a uniform RTSP stream internally.

15.1 Ingest normalization



A second output path runs in parallel to the one shown above: MediaMTX also serves the same (or, for the outward-facing per-camera streams produced after detection, the annotated) video as WebRTC, which the operator-facing dashboard consumes for live viewing in a browser without requiring any special plugin or native client.

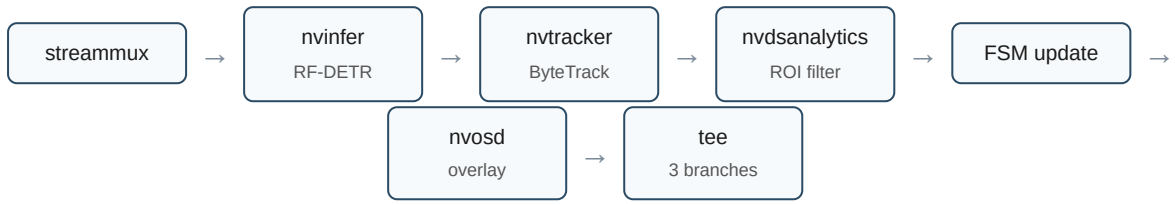
15.2 Automatic path provisioning

When an operator adds or edits a camera through the WasteWatch dashboard, the backend API calls MediaMTX's REST API to automatically create or update the corresponding ingest path — the operator never needs to manually edit a MediaMTX configuration file. This closes the loop between "operator adds a camera in the UI" and "that camera's stream is flowing into the detection pipeline" without a manual infrastructure step in between.

Chapter 16. DeepStream Pipeline Internals

The pipeline container is the technical heart of WasteWatch. It is built on NVIDIA DeepStream, a GStreamer-based framework purpose-built for constructing GPU-accelerated video analytics pipelines. This chapter walks through every stage of that pipeline in the order frames actually pass through it.

16.1 The full chain



16.2 Stream batching (streammux)

Because the system is designed to monitor between roughly ten and fifty camera feeds concurrently on a single GPU, individual frames from every active camera are first batched together by the streammux element into one combined GPU buffer per processing tick. This batching is what makes it economically feasible to run a large neural network across dozens of camera feeds on a single GPU: the detector below runs once per batch, amortizing its fixed per-inference overhead across every camera in that batch rather than paying that overhead once per camera per frame.

16.3 Object detection (nvinfer / RF-DETR)

The batched frames are passed to an nvinfer element running a detection model based on the RF-DETR architecture, compiled to a TensorRT engine for GPU-optimized inference at FP16 precision. The detector is deliberately restricted to a small, curated set of object classes relevant to the use case — principally people and vehicles — rather than the full range of object categories such a model is capable of recognizing. This class filtering happens at the pipeline configuration level, meaning the detector's output already excludes irrelevant object types before any tracking or zone logic runs, reducing both false-positive load on later stages and the amount of data that needs to be considered per frame.

Each detection carries a bounding box, a class label (e.g., "person" or "vehicle"), and a confidence score. Only detections above a configurable per-class confidence threshold are passed forward.

16.4 Multi-object tracking (nvtracker / ByteTrack)

Raw per-frame detections are not, by themselves, useful for measuring how long someone has been present in an area — a detector has no memory between frames. The nvtracker element, running the ByteTrack algorithm, solves this by matching detections across consecutive frames using a Kalman-filter-based motion model combined with a Hungarian-algorithm assignment step, producing a persistent numeric **track ID** for each object as long as it remains visible (even through brief occlusions). This is the identifier referenced throughout this document as the system's only concept of "who" is present — it carries no personal identity information and exists only for the duration a given object stays in frame.

> Because the track ID is generated fresh for each object's appearance in the frame and discarded when it leaves, ByteTrack provides continuity *within a single visit* to a monitored area, not identity continuity across separate visits, days, or camera views. This is a structural privacy property of the tracking algorithm itself, not merely a policy choice layered on top of it.

16.5 Region-of-interest filtering (nvdsanalytics)

Each camera has one or more restricted zones defined as polygons drawn by the operator over the camera's field of view. The nvdsanalytics element performs a point-in-polygon test on each tracked object's position, and only objects whose position falls inside a configured zone are passed forward into the event state machine described in Chapter 17. Objects detected and tracked anywhere

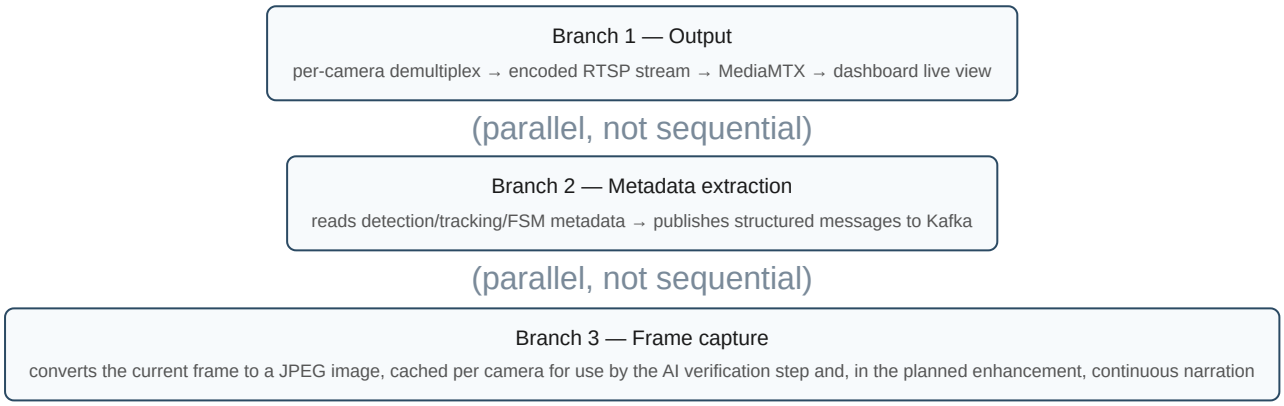
else in the frame are still tracked (so that, for example, a track already inside a zone that briefly steps just outside its boundary is not immediately lost) but never generate zone-entry events.

16.6 On-screen overlay (nvosd)

Before the frame reaches the branching point described below, an overlay stage draws bounding boxes and status labels onto the video for the operator-facing live view. A dedicated probe attached ahead of this stage colors and labels each box according to that object's current state in the event FSM (Chapter 17) — for example, an object in the OBSERVE state might render differently from one that has escalated to VIOLATION — giving an operator watching the live feed an at-a-glance read of the system's own internal assessment, not just raw detection boxes.

16.7 The three-way tee

After overlay, the single processed stream splits into three independent branches that run in parallel without blocking one another:

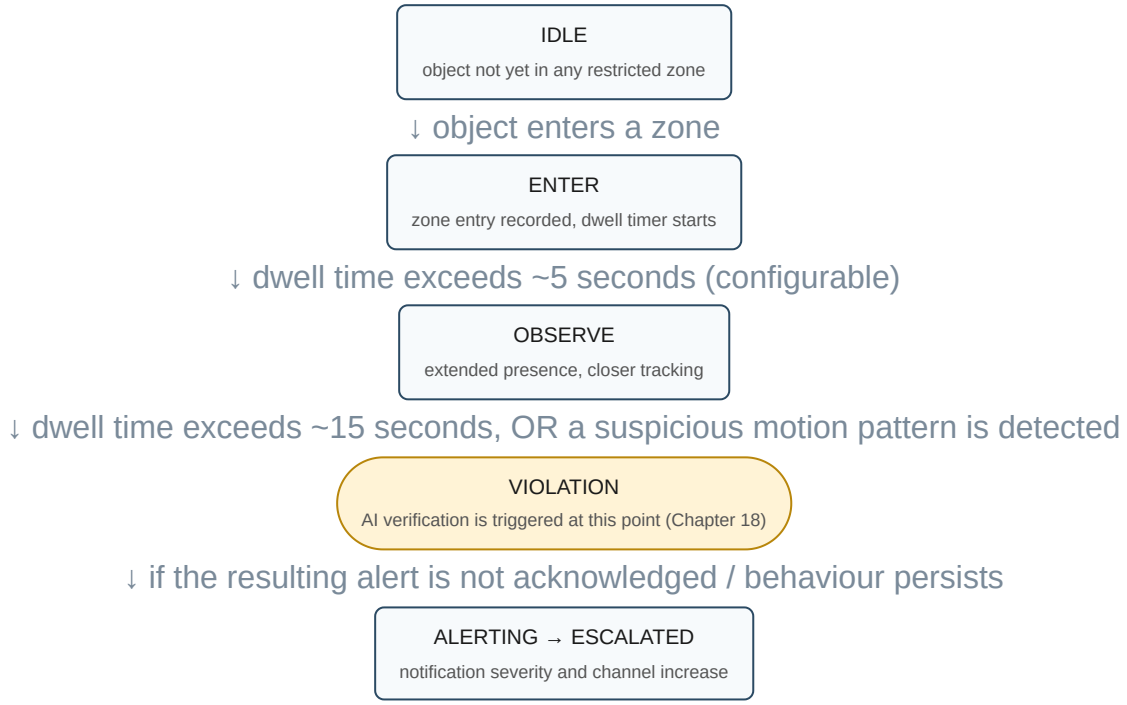


Each branch is deliberately configured so that if one branch stalls — for example, if a downstream network write slows down — it drops older buffered data rather than backing up and stalling the other two branches or the main detection chain. This "drop-old-data-under-pressure" behavior is a defensive design choice: a temporary slowdown in, say, writing to Kafka must never be allowed to freeze object detection across every camera in the system.

Chapter 17. Event FSM Deep Dive

The event finite-state machine (FSM) is the component that converts raw "an object is inside a restricted zone" signals from Chapter 16 into a graduated, time-aware assessment of whether that presence looks like a violation in progress. It runs once per tracked object, per frame, for as long as that object remains inside a monitored zone.

17.1 States and transitions



A separate, parallel exit path exists at every stage: if the tracked object leaves the restricted zone before crossing the next threshold, the state machine transitions to **EXITED** and the object's session ends without a violation record being created. The large majority of zone entries recorded by the system — someone briefly walking past or standing momentarily near a bin — end here, with no event generated at all.

17.2 What each transition carries

Every state transition is recorded with the camera identity, the track ID, the zone/dwell-time context, the object's class (person or vehicle), and a summary of its recent movement path. This context travels with the event all the way through to the AI verification step, where it is used to construct the natural-language prompt sent to the vision-language model (Chapter 18) — the model is told, in effect, "this object has been in this zone for this long, moving in this pattern," not just shown a bare image with no context.

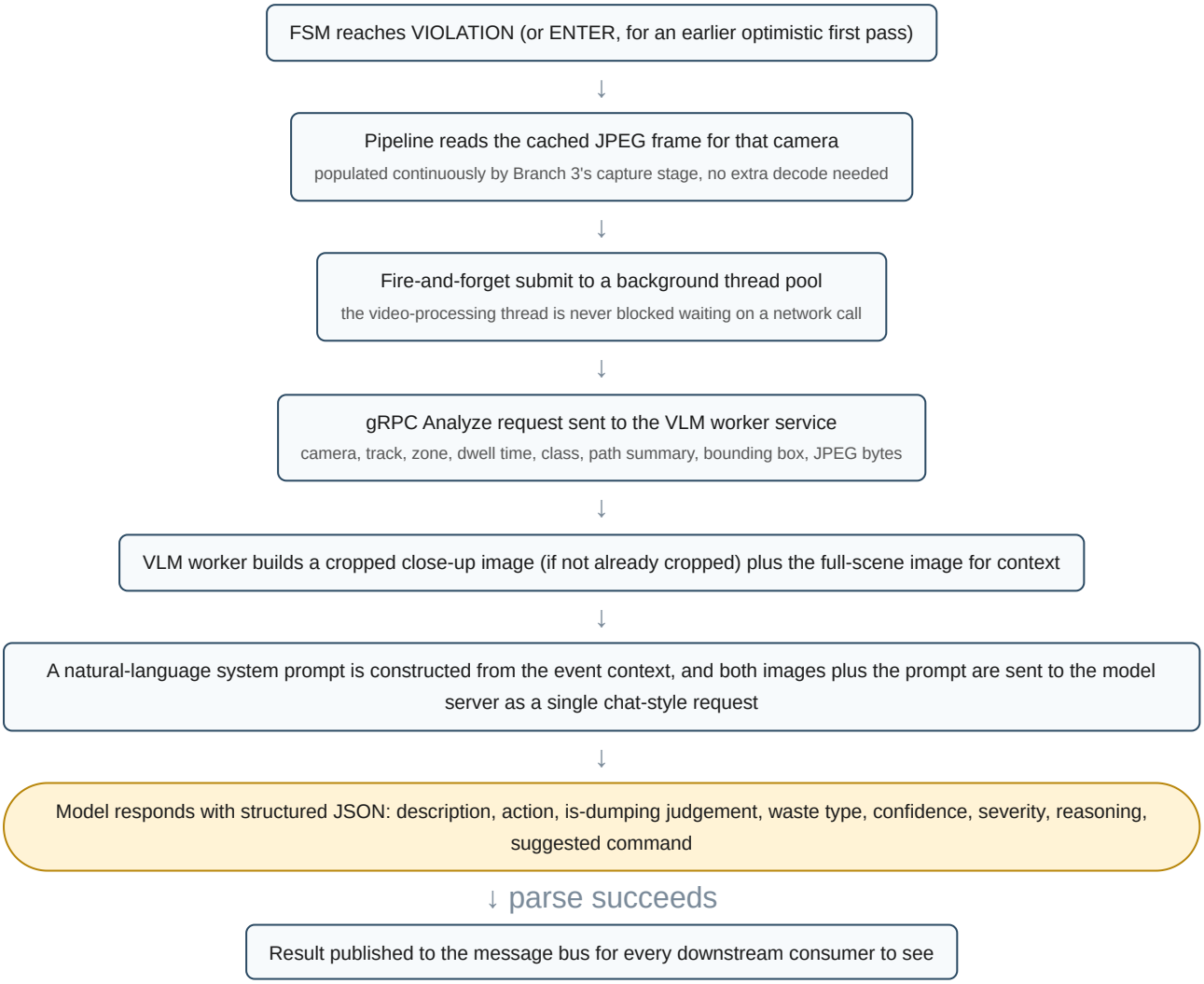
17.3 Configurability

The dwell-time thresholds shown above (5 seconds for ENTER→OBSERVE, 15 seconds for OBSERVE→VIOLATION) are operator-configurable defaults, not hard-coded constants. A site with, for example, a bus stop inside or adjacent to a restricted zone would reasonably configure longer thresholds than an isolated alley where any lingering at all is unusual. This configurability is exposed through the operator dashboard and does not require a software change to adjust per camera or per zone.

Chapter 18. VLM Trigger Path Deep Dive

This chapter documents, at full technical depth, what happens the moment a tracked object's state machine reaches VIOLATION. This is the path that is live in production today.

18.1 Full call sequence



If the model's response cannot be parsed as valid structured output, the system retries once with a stricter instruction, and if that also fails, publishes a safe fallback result (zero confidence, informational severity, "monitor" as the suggested action) rather than crashing or silently dropping the event. A failed AI call never fails silently — it degrades to "keep watching," never to "assume the worst."

18.2 Request and response shapes

The request the pipeline sends carries: camera identity, track identifier, a session identifier combining the two, the zone name, the measured dwell time in seconds, the detected object class, a short textual summary of the object's recent movement path, the bounding box that triggered the analysis, the JPEG-encoded frame(s), corresponding timestamps, and a unique request identifier for traceability.

The model's response, once parsed, carries: a natural-language description of the person or object, a short label for the action being performed, a boolean judgement of whether dumping appears to be occurring, an estimated waste-type category, a numeric confidence score, a severity rating, the model's own stated reasoning for its judgement, a suggested next action (for example, "alert" versus "monitor"), and — where relevant — a natural-language deterrent message suitable for playback through an on-site speaker.

18.3 Reliability behavior

Two specific reliability mechanisms protect this path from degrading overall system performance during a partial outage of the AI verification service. First, the gRPC client tracks consecutive failures and, after five in a row, opens a thirty-second circuit breaker during which further requests are silently skipped rather than repeatedly attempted against a service that is clearly down — this prevents a stalled or crashed model server from causing a backlog of stuck requests to accumulate. Second, because the request itself is submitted fire-and-forget from a background thread pool, even a slow (rather than fully failed) model response cannot delay the processing of the next video frame; detection and tracking continue at full speed regardless of how long AI verification is taking for a previously-flagged object.

Chapter 19. MiniCPM-o 4.5 Model Internals and Omni-Flow

This chapter documents the model planned to replace the current vision-language model, and — because it is the same model that will power the planned continuous-narration capability described in Chapter 20 — its internal streaming architecture in detail.

19.1 Model composition and resource footprint

Property	Value
Total parameters	Approximately 9 billion
Vision encoder	SigLip2
Audio encoder	Whisper-medium (not used by WasteWatch — vision-only deployment)
Speech decoder	CosyVoice2 (not used by WasteWatch)
Language backbone	Qwen3-8B
VRAM — full precision (BF16)	Approximately 19 GB
VRAM — 4-bit quantized	Approximately 11 GB

> At the time of writing, 4-bit-friendly quantization formats (AWQ/GPTQ) commonly used to shrink the currently deployed model are not confirmed to be available for this newer model. Deployment will therefore run at either full BF16 precision or an available quantized variant once verified, and GPU capacity planning must account for the larger of these two footprints until confirmed otherwise.

19.2 Two distinct ways the model is called

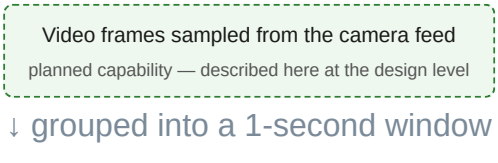
A critical technical detail — easy to overlook because both uses involve "sending an image to the model" — is that the trigger path (Chapter 18) and the planned narration path (Chapter 20) do not call the model the same way at all:

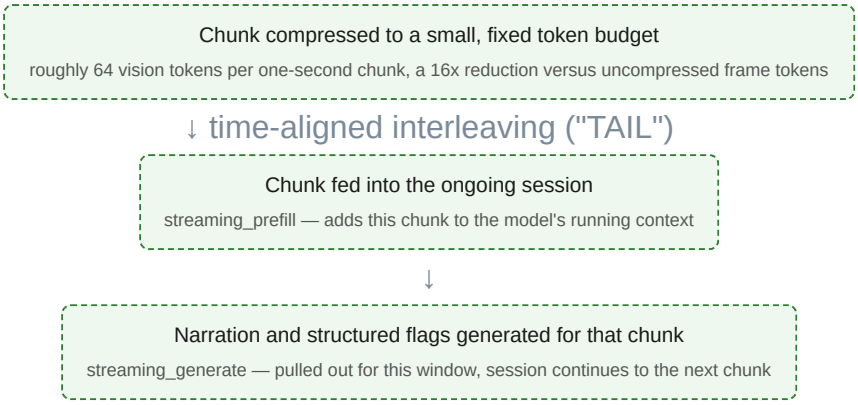
Aspect	Trigger path	Narration path (planned)
Call shape	Stateless, one-shot request/response, styled like a standard chat message	Stateful, long-lived streaming session held open per camera
Context across calls	None — every request is independent	Continuous — the model accumulates context across the whole session
Serving path used	Ordinary chat-completions-style endpoint	The model's native duplex streaming interface

The model server that will host this model — a specialized extension of the standard serving framework built specifically to support this family of "omni" models, distinct from the mainline serving software the currently deployed model uses — exposes both interfaces, which is why one underlying model and one underlying server process can support both use cases without running two separate deployments.

19.3 The Omni-Flow streaming framework

The model's continuous, real-time understanding capability is built on a framework, referred to by its designers as "Omni-Flow," that aligns incoming video (and, where enabled, audio) to a shared timeline broken into fixed one-second chunks, rather than processing an arbitrary, irregular stream of frames.





The choice of a one-second chunk is not arbitrary: the model's designers found that much shorter windows (on the order of a tenth to a fifth of a second) starve the model of enough visual context per step to produce reliable output, making one second the practically validated minimum. The "TAIL" (time-aligned interleaving) mechanism exists to prevent the model's narration output from gradually drifting out of sync with what is actually happening on camera as a session runs for an extended period — a known failure mode in naive streaming designs where output pacing and input pacing slowly diverge.

Chapter 20. Planned Narration Path Deep Dive

> Everything in this chapter describes a designed but not-yet-deployed capability. It is included at full technical depth because the design has been fully specified and approved internally, and because a reviewing authority evaluating this system's future direction needs the same level of detail as for the live trigger path in Chapter 18. No component described here processes live camera data today.

20.1 Why a second, independent path exists

The trigger path described in Chapter 18 depends entirely on the tracking-and-dwell-time logic in Chapter 17 deciding that an object's presence looks suspicious. This structurally cannot catch violation patterns that do not depend on lingering — for instance, someone briefly throwing an item over a fence while walking past without stopping, or a vehicle passing through with a visibly uncovered or overflowing load, neither of which necessarily triggers a meaningful dwell time. The planned narration path addresses this gap by having the model watch continuously, independent of any specific object's tracked dwell time.

20.2 Full planned sequence

A recurring timer (independent of any FSM state) fires at a configurable sampling rate, e.g. once per second, for every actively monitored camera

↓

The most recently captured frame for that camera (the same cached frame Branch 3 in Chapter 16 already produces) is read — no new decoding work is added to the pipeline

↓

The frame is submitted to that camera's dedicated, long-lived narration session (Chapter 19.3)

↓

The model returns a running natural-language narration plus zero or more structured flags for that window (e.g., "dumping motion," "uncovered load," or an open-ended "anomaly" category)

↓ no flags

Nothing is recorded; the loop continues to the next sample

↓ one or more flags present

A supporting evidence clip is captured, and the flag is published to a dedicated message-bus topic reserved for narration-derived observations

↓

A dedicated consumer resolves the camera and inserts a record into a separate database table, explicitly marked as pending human review

↓

An authorised reviewer confirms (which creates a full violation record, entering the exact same downstream evidence/alert flow as a trigger-path violation) or dismisses (which closes the item with no record created)

20.3 Why this cannot bypass the review requirement

Unlike the trigger path — where the AI's judgement, together with the preceding dwell-time signal, is treated as sufficient to raise an alert directly — the narration path's output is, by design, never written into the same table as a confirmed violation without an

explicit human confirmation action in between. This is enforced at the data-model level, not merely by internal process convention: the pending record and the confirmed violation record are different rows in different tables, connected only by the reviewer's confirm action, which itself is logged with the reviewer's identity and timestamp.

20.4 GPU sharing between the two paths

Both the trigger path and the narration path, once deployed, will share the same model server process and the same GPU. The serving framework's own request-batching capabilities allow short trigger-path requests and long-lived narration sessions to interleave on the GPU rather than strictly queuing one behind the other. The design nonetheless includes a configurable cap on the total number of simultaneous narration sessions specifically so that an unbounded number of always-on narration sessions cannot starve the latency-sensitive trigger path of GPU time during high-traffic periods — this cap is a deployment-time tuning parameter to be set based on real load testing, not a value fixed in the design itself.

Chapter 21. Kafka Messaging Layer

Every asynchronous message in WasteWatch — detection metadata, state-change events, AI analysis requests and results, alerts, plate reads, generated speech-audio jobs, evidence-recording triggers, structured logs, live configuration updates, phone-escalation requests, and camera health signals — passes through a single message-bus service (Apache Kafka), organized into named topics that consumers subscribe to individually.

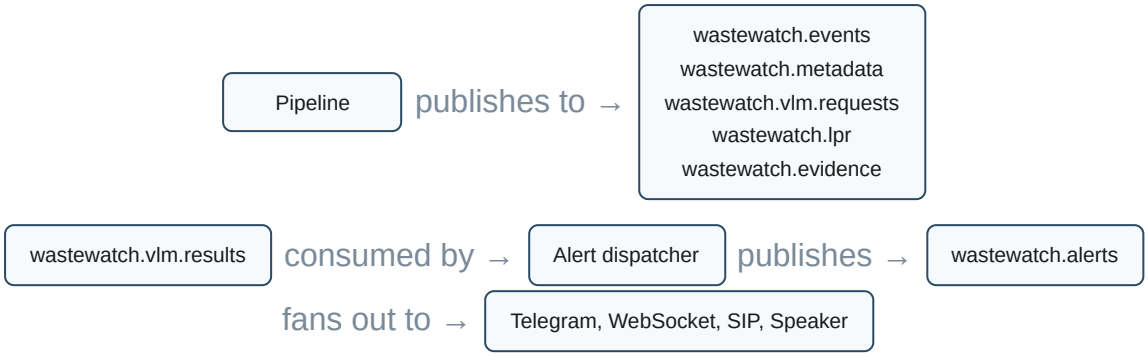
21.1 Topic reference

Topic	Carries	Published by
wastewatch.metadata	Per-frame detection/tracking metadata	Pipeline
wastewatch.events	FSM state-change events (ENTER, OBSERVE, VIOLATION, EXITED, etc.)	Pipeline
wastewatch.vlm.requests	Legacy/alternate path for requesting AI analysis via the message bus rather than direct gRPC	Pipeline
wastewatch.vlm.results	Structured AI verification results	VLM worker
wastewatch.alerts	Fan-out-ready alert events for notification channels	Alert service
wastewatch.lpr	Vehicle plate-recognition requests	Pipeline
wastewatch.tts	Speech-audio generation requests	Pipeline / alert service
wastewatch.evidence	Evidence file save/sync requests	Pipeline / dispatcher components
wastewatch.logs	Structured application logs	All services
wastewatch.config.updates	Hot-reloadable configuration changes (e.g., ROI edits, thresholds)	API
wastewatch.sip	Phone-call escalation requests	Alert / escalation logic
wastewatch.camera.status	Camera connectivity/health signals	Health monitor
wastewatch.narration.flags *(planned)*	Narration-derived anomaly flags pending human review	Pipeline (narration sampler, once deployed)

21.2 Consumer group reference

Consumer group	Subscribes to	Role
Alert dispatcher group	wastewatch.vlm.results	Applies throttling, decides notification channels, publishes to wastewatch.alerts
Escalation group	wastewatch.alerts / repeat-offense signals	Raises severity and triggers phone escalation for persistent cases
Evidence worker group(s)	wastewatch.evidence, wastewatch.events	Persists snapshots/metadata; starts and stops smart video recording on VIOLATION/EXITED
LPR worker group	wastewatch.lpr	Runs plate recognition, writes results to the database
Log ingestion group	wastewatch.logs	Writes structured logs into the searchable log store
Narration worker group *(planned)*	wastewatch.narration.flags	Persists pending review-queue records

21.3 Fan-out pattern



This publish/subscribe structure is what allows the system to add a new downstream capability — for example, the planned narration review workflow — by introducing one new topic and one new consumer, without touching the pipeline's core detection and tracking logic or any existing consumer's code.

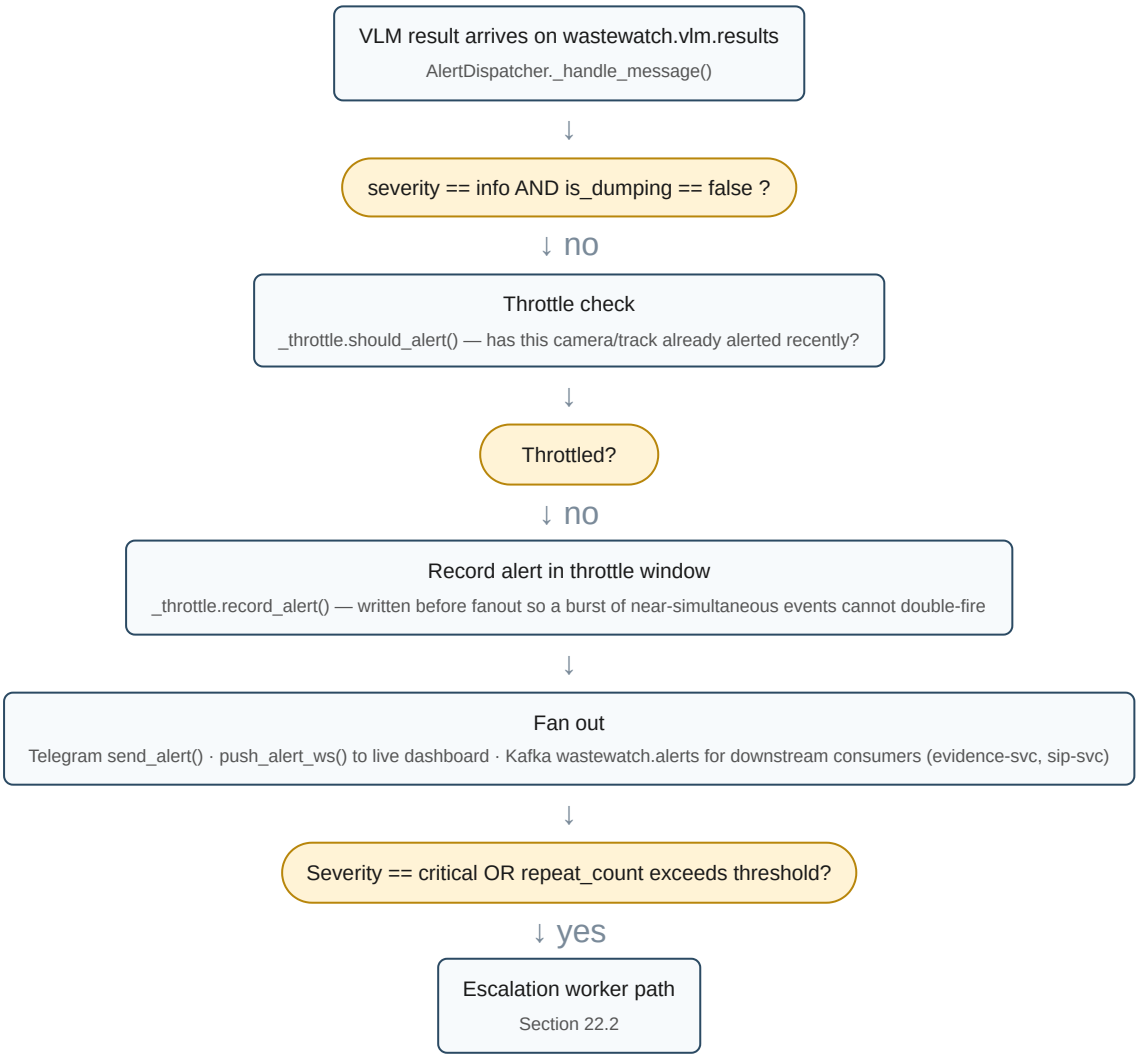
Chapter 22. Alert and Escalation Services

Detection and AI verification only have value if the resulting judgement reaches a human being who can act on it. The alert and escalation layer is the subsystem responsible for that last mile: taking a confirmed VLM result off the Kafka bus and turning it into a notification on a dashboard, a message on a phone, or — in the more severe and repeat cases — an actual phone call demanding attention. This chapter documents the two services that make up this layer: `alert-svc`, which performs the moment-of-decision fanout, and its internal escalation-worker component, which watches for patterns across time rather than single events.

22.1 The Alert Dispatcher

The alert dispatcher lives in `backend/app/alerts/dispatcher.py` as the `AlertDispatcher` class. It is a Kafka consumer subscribed to `wastewatch.vlm.results` — the topic that both the trigger-based VLM path and, once the planned narration-confirmation flow lands, human-confirmed narration flags ultimately feed into as fully-fledged violation events. Every message that arrives on this topic passes through a single entry point, `_handle_message()`, which is the dispatcher's decision core.

The logic inside `_handle_message()` is deliberately conservative about what counts as alert-worthy. A message is only escalated toward the human-facing channels if the reported severity is above the lowest tier and the model's own `is_dumping` field is true. A response classified as `severity: info` and `is_dumping: false` — which is the expected outcome for the majority of ordinary passers-by who happen to dwell slightly too long in a restricted zone — is logged and dropped without ever reaching a person. This threshold exists precisely to prevent alert fatigue: if every borderline VLM call generated a notification, operators would quickly learn to ignore the channel altogether, defeating its purpose.



Two structural properties of this design are worth calling out explicitly, because they represent deliberate engineering trade-offs rather than incidental behaviour:

- **Recording precedes fanout.** The throttle window is updated by calling `record_alert()` before the Telegram/WebSocket fanout is executed, not after. This ordering exists so that if the fanout call itself is slow or partially fails, the system does not end up re-evaluating the same event as "not yet alerted" and sending a duplicate notification a few seconds later. Idempotency is enforced at the point of decision, not at the point of delivery.
- **Fanout failure is isolated per channel.** The Telegram client and the WebSocket push are invoked as independent calls rather than a single blocking chain — a Telegram API outage does not prevent the live dashboard from updating, and vice versa. This is standard fan-out resilience practice: no single downstream channel can become a single point of failure for the others.

> The dispatcher's throttle logic exists specifically to protect operators from alert fatigue, which is a well-documented failure mode in monitoring systems: when notification volume exceeds what a human can meaningfully triage, operators begin ignoring the channel wholesale, and genuine high-severity events get lost in the noise along with the false positives. Throttling per camera/track combination ensures a single lingering individual does not generate a new alert on every VLM re-check.

22.2 Escalation Worker — Repeat-Offense Handling

`backend/app/alerts/escalation_worker.py` runs as a separate Kafka consumer group (`GROUP_ESCALATION`) reading the same event stream, but with a different responsibility: it is not deciding whether a single event is alert-worthy, it is deciding whether a *pattern* of events at the same location warrants a different response channel entirely. The distinction matters because a single dumping event and a location that has now seen five dumping events in the past hour are not the same operational problem — the first needs an alert; the second needs someone physically dispatched, or at minimum a phone call that cannot be silently dismissed the way a chat notification can.

The `ViolationEvent` database row carries a `repeat_count` field precisely to support this distinction (see Chapter 27 for the full schema). When the escalation worker observes that a camera/zone combination has accumulated repeat violations within a rolling window, it raises the severity of subsequent notifications and directs them toward the phone-call escalation channel (`sip-svc`, detailed below) rather than leaving them at dashboard/Telegram level only.

22.3 Downstream Channels

Channel	Trigger condition	Behaviour
Operations dashboard (WebSocket)	Every non-suppressed alert	`push_alert_ws()` pushes the alert payload to any connected dashboard client in real time via the `ws_alerts` WebSocket router in the API container. This is the lowest-latency, lowest-friction channel and is always populated regardless of severity tier, giving operators a complete audit trail of everything the system considered alert-worthy, not only what was escalated further.
Telegram bot	Every non-suppressed alert (severity-dependent formatting)	A structured message including camera identity, timestamp, the VLM's description and confidence, and — where available — the annotated snapshot image, sent to configured chat IDs (`settings.telegram_chat_ids`).
SIP / phone call escalation	Critical severity or repeat-offense pattern from the escalation worker	Routed through `sip-svc`, which is configured against either a self-hosted SIP gateway (`settings.sip_server` / `sip_username` / `sip_domain`) or Twilio (`settings.twilio_account_sid` / `twilio_auth_token` / `twilio_from_number`) depending on deployment. Contacts to call are configured per escalation tier via `sip_escalation_contacts`.
On-site audio (IP speaker)	Configurable, typically all confirmed dumping events at	Publishes to `wastewatch.tts`; see Chapter 25 for the full text-to-speech pipeline. This is the only channel that produces an effect at the physical location itself, rather than in an operator-facing interface.

	a site with a speaker installed	

The dispatcher intentionally does not attempt to determine severity itself from raw detection data — severity is a field already assigned by the VLM's structured response (see Chapter 18/19 material on the VLM trigger path) or, for narration-derived events, assigned at the point a human confirms the flag (Chapter 5 narrative discussion, Part I). The dispatcher's role is strictly "given a severity and a dumping determination that has already been made upstream, decide who needs to know and how urgently" — it does not re-adjudicate the underlying finding.

Chapter 23. Evidence and Recording Subsystem

An alert without evidence is an accusation without proof. The evidence subsystem exists to ensure that every event the system considers significant enough to notify a human about is backed by a retrievable, timestamped record: a snapshot, a short video clip spanning the moments before and after the event, and the structured reasoning that led the system to flag it. This chapter describes how that evidence is produced, in two places — inside the pipeline container at the moment of detection, and inside the backend's evidence service as the event's lifecycle plays out — and how the two halves are reconciled into a single browsable history.

23.1 Pipeline-Side Evidence Writer

Evidence capture begins inside the `pipeline` container itself, in `pipeline/app/evidence_writer.py`. This module is deliberately co-located with the GStreamer processing loop because it needs access to raw frame data at the moment detection happens, before that data has been serialized, compressed for network transport, or discarded. Its lifecycle follows the same track-level lifecycle the FSM manages:

- `start_evidence()` — called when a track's FSM state first reaches a level warranting evidence capture, opens a new evidence session for that track/camera combination.
- `add_frame()` — called on subsequent frames while the track remains active, accumulating the frame sequence that will become the video clip.
- `capture_violation_keyframe()` — called specifically at the moment the FSM transitions to `VIOLATION`, capturing a single best-quality still image distinct from the rolling frame buffer — this is the image most likely to be shown to a human reviewer or included in a Telegram alert, so it is captured deliberately rather than sampled arbitrarily from the clip.
- `finalize()` — called when the track exits (FSM `EXITED` state), closes out the evidence session, writes the accumulated frames to an MP4 clip, and writes a JSON sidecar file (`evt_*.json`) containing the structured metadata: camera, track ID, timestamps, FSM state history, and (once the VLM result has returned) the AI's analysis.

These files are written under a directory structure the rest of the system agrees on by convention: `{WW_EVIDENCE_DIR}/{violations|suspicious}/{camera_name}/{YYYY-MM-DD}/evt_*.json`, with the corresponding media files (JPEG keyframe, MP4 clip) alongside. The date-partitioned, camera-partitioned layout is what makes the filesystem-scan approach used by the history browser (Section 23.3) practical even without a database index — an operator or the API can narrow a search to a specific camera and day without touching unrelated files.

23.2 Backend-Side Evidence Worker and Smart Recording

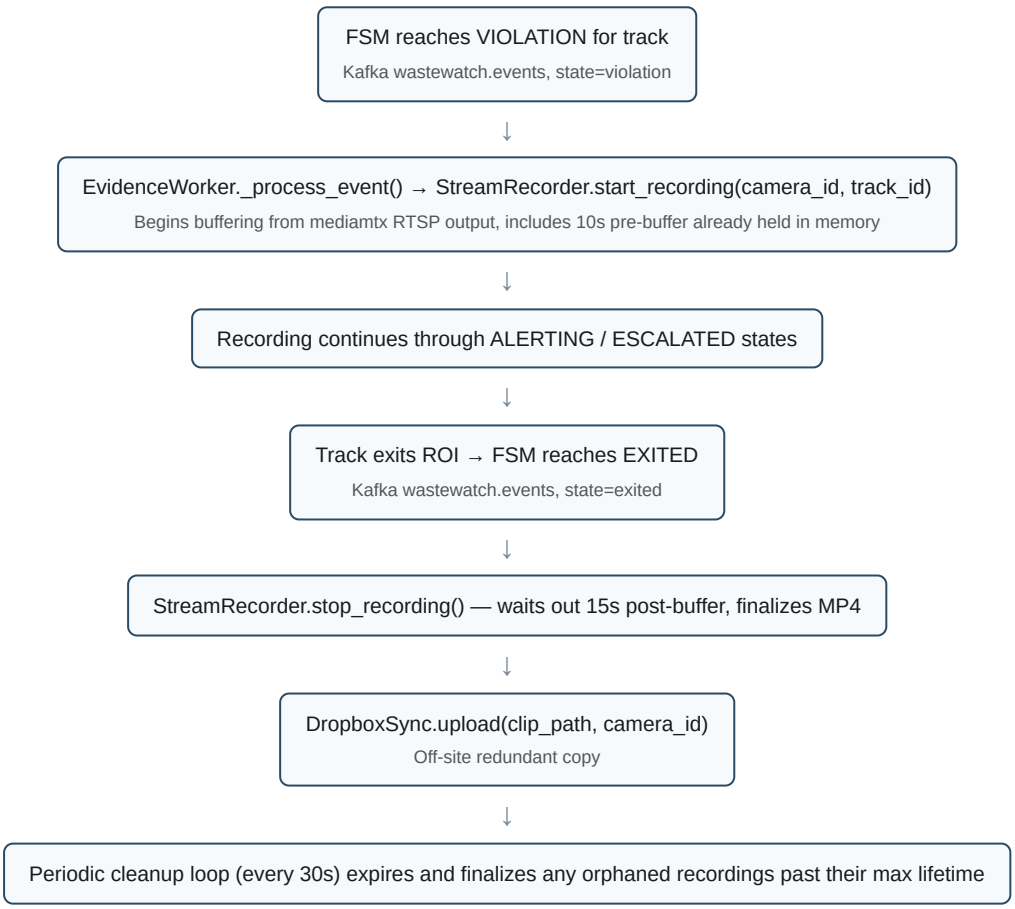
`backend/app/evidence/worker.py` defines `EvidenceWorker`, which runs two independent Kafka consumers under two separate consumer groups:

Consumer group	Topic	Responsibility
`wastewatch.group.evidence-worker`	`wastewatch.evidence`	Processes discrete evidence file requests — snapshot uploads and metadata records pushed explicitly onto this topic — via `_process_evidence()`, which dispatches to `EvidenceRecorder.save_snapshot()` or `save_metadata()` depending on message type, then syncs the result to Dropbox.
`wastewatch.group.evidence-recording`	`wastewatch.events`	Watches raw FSM state-change events (the same stream the pipeline's `event_fsm.py` publishes on every transition) for two specific transitions: `violation`, which starts a smart recording of the live inference output stream via `StreamRecorder.start_recording()`, and `exited`, which stops it via `stop_recording()`, retrieves the finished clip path, and pushes it to Dropbox.

The separation into two independent consumers, on two independent Kafka topics, reflects a genuine architectural distinction: the evidence-worker path handles explicit, one-shot "please save this" requests, whereas the evidence-recording path is a continuous, stateful process that has to track which cameras currently have an open recording session and correctly close it out even under pipeline restarts or partial failures — which is why `StreamRecorder` maintains its own internal state (a mapping of camera/track to in-progress recording) rather than being purely reactive to individual messages.

23.2.1 Pre-buffer / Post-buffer Recording

`StreamRecorder` is configured with a `pre_buffer_s` of 10 seconds and a `post_buffer_s` of 15 seconds. This means the clip ultimately saved for a violation does not begin at the moment the FSM reaches `VIOLATION` — it begins 10 seconds *before* that moment, capturing the lead-up to the event (how the person approached, what they were carrying, whether anyone else was present), and continues 15 seconds *after* the track exits, capturing the aftermath (did they leave the waste behind, did anyone else react). This pre/post buffering is what makes the resulting clip usable as evidence in the way a bare "moment of detection" snapshot alone would not be — reviewers and, if the case escalates, enforcement personnel, need the surrounding context to judge intent and outcome, not just a single frame.



A background `_cleanup_loop()` runs on a 30-second cycle specifically to guard against the failure mode where a track's `exited` event is lost or delayed — without this safeguard, a recording session could theoretically remain open indefinitely, consuming storage and never being finalized into a retrievable clip. The cleanup loop enforces a maximum recording lifetime and force-finalizes anything that exceeds it, logging the recovered path.

23.3 The History Browser API

Operators and, where authorized, external reviewers access accumulated evidence through `backend/app/api/routes/evidence.py`, which exposes three endpoints reflecting the two different storage mechanisms described above:

Endpoint	Backing store	Behaviour

`GET /evidence`	Postgres `evidence_files` table	The original, database-backed listing — returns rows from the `EvidenceFile` model (Chapter 27), each linked to a `ViolationEvent` via foreign key, filterable by `event_id`.
`GET /evidence/files`	Filesystem scan under `WW_EVIDENCE_DIR`	Recursively scans for `evt_*.json` sidecar files written directly by the pipeline's `evidence_writer.py`, resolves each record's plain-text camera name to its database UUID via a live lookup against the `Camera` table (so the frontend can cross-reference evidence with alerts, which are stored by UUID), and returns results newest-first. This is the endpoint the frontend's `TimelineView` component actually calls for the day-to-day history browsing experience.
`GET /evidence/file?path=...`	Direct filesystem read	Serves a single evidence file (an MP4 clip or JPEG keyframe) by absolute path. The path is resolved and checked against `EVIDENCE_ROOT` before serving — a request for a path outside the evidence root is rejected with a 403, preventing path-traversal access to arbitrary files on the host (see Chapter 29 for the full security treatment of this guard).

The coexistence of a database-backed evidence table and a filesystem-scan endpoint reflects the system's evolution over time rather than a single unified design choice made from scratch — the filesystem-based sidecar approach was added later specifically to support the richer, pipeline-native evidence capture (pre/post buffer clips, FSM state history) without requiring every pipeline-side write to also perform a synchronous database insert from inside the latency-sensitive GStreamer processing loop. Both paths remain in use.

Chapter 24. License Plate Recognition Subsystem

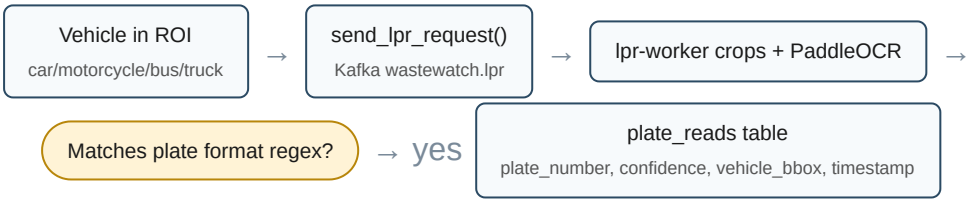
Illegal dumping frequently involves a vehicle — a truck or car used to transport and unload waste at an unauthorized site — and in these cases, the vehicle's license plate is often the single most actionable piece of evidence for follow-up enforcement, since a person's identity may be unknown but a plate can be traced to a registered owner. The LPR (License Plate Recognition) subsystem exists specifically to capture and read plates for exactly this class of event, without running plate recognition indiscriminately on every vehicle that passes any camera.

24.1 Triggering Conditions

LPR is deliberately gated to only the vehicle classes the detector is configured to recognize: car, motorcycle, bus, and truck (see the detection class filter table in the system overview, Chapter 13/16). A pedestrian detection never triggers an LPR request. When a tracked vehicle of one of these classes is confirmed inside a restricted ROI, the pipeline publishes an LPR request to Kafka via `kafka_producer.py: send_lpr_request()`, carrying the camera ID, track ID, the vehicle's bounding box, and its class.

24.2 Processing Pipeline

`backend/app/lpr/worker.py` runs as a Kafka consumer under group `wastewatch.group.lpr-worker`, subscribed to the `wastewatch.lpr` topic. For each request, it crops the vehicle bounding box from the relevant frame, runs it through PaddleOCR (chosen specifically for its GPU-accelerated inference and strong out-of-the-box performance on non-Latin and mixed-format plates, which matters for Indian vehicle registration formats), and applies a regex-based validation step against expected Indian license plate formatting before accepting a reading as valid. A read that does not match the expected format pattern is discarded rather than stored, since a malformed "plate" string is more likely to be OCR noise than a genuine misprint.



Valid readings are written to the `plate_reads` table (see Chapter 27 for the full schema), linked where possible to the originating `track_session_id` so that a plate read can be cross-referenced against the violation event it was captured alongside during a later review.

Chapter 25. Text-to-Speech and On-Site Audio Subsystem

Where an IP speaker is installed at a monitored site, the system can play an audible deterrent message in real time — the intent being that immediate, on-the-spot feedback ("this area is under camera surveillance, please do not dispose of waste here") can itself prevent a violation from being completed, which is a materially better outcome than only detecting and penalizing it after the fact.

25.1 From VLM Response to Spoken Audio

The VLM's structured response schema includes a `deterrent_message` field — a short, context-appropriate spoken message generated as part of the same model call that determines whether dumping is occurring (see Chapter 18/19 for the full request/response schema). When a response with a non-empty deterrent message and sufficient severity is produced, the pipeline (or, depending on deployment configuration, the alert dispatcher) publishes it to `wastewatch.tts` via `kafka_producer.py: send_tts_request()`, carrying the camera ID, track ID, the message text, and a priority tag.

The `tts-worker` / `tts` containers consume this topic and generate the actual audio using Piper, a lightweight, fully local, ONNX-based text-to-speech engine chosen specifically because it runs efficiently on CPU (freeing the GPU entirely for detection and VLM inference) and requires no external API call — consistent with the system's broader design principle that no operationally sensitive data (in this case, the fact that a specific message was played at a specific camera at a specific time) leaves infrastructure the operator controls.

> Generated audio is cached — the `tts_cache` table (Chapter 27) stores a hash of previously synthesized text alongside the resulting audio URL and duration, so that a commonly repeated deterrent message does not require re-synthesis on every occurrence, reducing both CPU load and end-to-end latency between detection and the audio actually playing.

25.2 Delivery

Generated audio is pushed to two destinations simultaneously: the physical IP speaker at the relevant camera location (configured per-camera in `settings.ip_speaker_configs`, gated by `ip_speaker_enabled`), and the operator-facing WebSocket channel, which allows a remote operator to hear — and see a transcript of — exactly what was played on-site, in real time, supporting oversight of the automated audio channel rather than leaving it as a silent, unmonitored action.

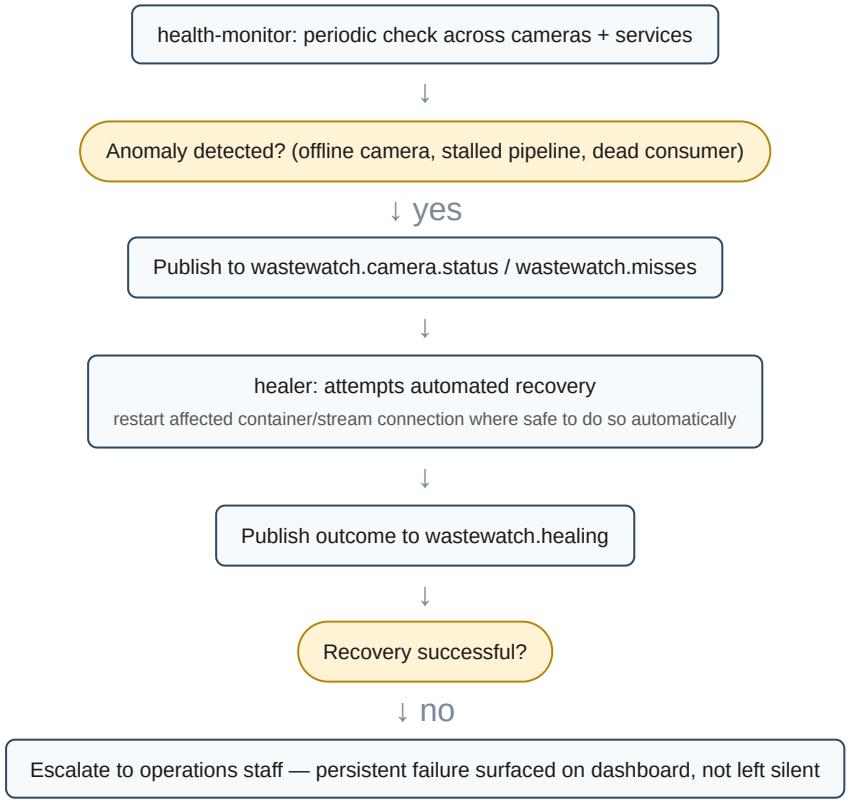
Chapter 26. Health Monitoring and Self-Healing

A detection system that silently stops working is more dangerous than one that never existed, because it creates a false sense of coverage — an area believed to be monitored is, in fact, unwatched, and nobody knows to compensate. The health monitoring subsystem exists specifically to prevent this failure mode: every camera feed and every internal processing component is continuously checked, and any gap in coverage is surfaced immediately rather than discovered only when someone happens to review footage and notices a hole.

26.1 What Is Monitored

- **Camera connectivity** — whether each configured RTSP/RTMP/HTTPS source is actively delivering frames through MediaMTX. A camera that has gone offline, lost network connectivity, or is delivering a frozen/corrupted stream is detected via the pipeline's connection monitor, which publishes state changes to `wastewatch.camera.status`.
- **Pipeline processing health** — whether the DeepStream pipeline's internal probes (detection, FSM update, capture) are actually executing on a healthy cadence, versus stalled (a known historical failure mode, documented directly in the pipeline's own code comments regarding queue backpressure — see the `leaky=downstream` queue configuration discussed in Chapter 16, which exists specifically because a stalled output branch was once found to silently freeze the entire metadata extraction path).
- **Downstream service liveness** — whether Kafka consumers for each service (alert dispatcher, evidence worker, LPR worker, log worker) are actively processing their assigned partitions rather than having silently crashed or hung.

26.2 Health-Monitor and Healer Containers



The distinction between the `health-monitor` and `healer` containers is one of detection versus remediation: `health-monitor`'s job is purely to observe and report, while `healer` is the component authorized to take corrective action (restarting a stream connection, restarting a stalled service container) where such action is judged safe to perform without human sign-off. This separation follows a standard operational-safety principle — the component that decides "something is wrong" is not the same component empowered to act on that decision unilaterally beyond a bounded, pre-approved set of remediation actions, and any failure the automated healer cannot resolve is explicitly escalated rather than silently retried indefinitely.

Chapter 27. Backend API and Database Schema

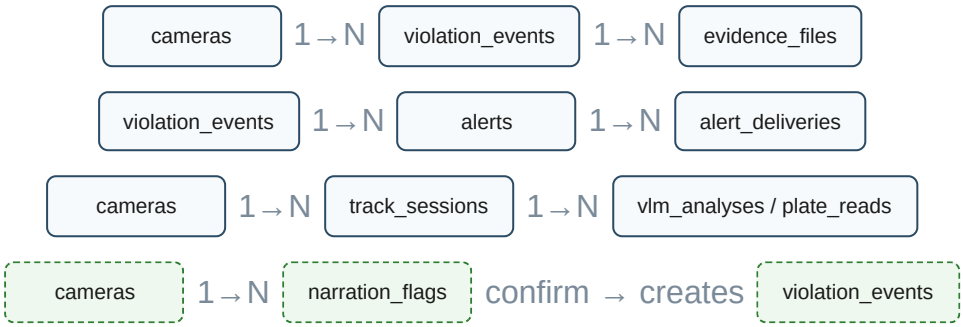
This chapter provides the full reference for the relational schema underpinning the system's persistent record — every table, its purpose, and its relationships to the others — along with an overview of the API route groups built on top of it. Where Part I described these records in plain operational language (a "violation record," a "review queue entry"), this chapter is the literal, column-level specification.

27.1 Core Tables

Table	Purpose and key columns
`cameras`	One row per configured camera: name, connection details, `roi_polygons` (JSONB — the restricted-zone polygon(s) drawn by the operator), `recording_config`, `is_pinned` (dashboard display preference), `last_event_at`.
`violation_events`	The core violation record. Columns: `camera_id` (FK → cameras), `timestamp`, `event_type`, `severity`, `confidence`, `track_id`, `dwell_time_s`, `vlm_response` (JSONB — the full structured AI response, preserved verbatim for audit purposes), `person_id` (only ever a session-scoped tracking identifier, never a real-world identity — see Chapter 8's privacy discussion), `repeat_count`, `snapshot_url`, `video_clip_url`, `status` (active/acknowledged/etc.), `acknowledged_by`, `acknowledged_at`, `notes`, `is_valid` (nullable — populated when a human reviewer explicitly confirms or rejects the event).
`alerts`	A notification record distinct from the violation event itself — one violation can, in principle, generate alert records across multiple channels. Columns: `event_id` (FK → violation_events, nullable), `camera_id` (FK → cameras, nullable), `severity`, `title`, `message`, `zone`, `channel`, `tts_audio_url`, `delivered_at`, `status`.
`alert_deliveries`	Per-channel delivery attempt record for a given alert: `alert_id` (FK → alerts), `channel`, `status`, `payload`, `delivered_at`, `error_message`, `retry_count` — this table is what makes a delivery failure (e.g. a Telegram API timeout) auditable and retryable rather than silently swallowed.
`track_sessions`	One row per FSM lifecycle for a tracked object: `camera_id`, `track_id`, `fsm_state` (enum: enter/observe/violation/alerting/escalated/exited), `entered_at`, `exited_at`, `dwell_time_s`, `bbox_history` (JSONB — the full path the object took through the frame, supporting later path-summary analysis).
`vlm_analyses`	A durable, database-backed record of every VLM call and its result, independent of whether it resulted in an alert: `track_session_id` (FK), `camera_id`, `frames_analyzed`, `prompt_used`, `response` (JSONB), `inference_time_ms`, `model_version` — this last field is what allows a later audit to determine exactly which model version (Qwen2.5-VL-7B-AWQ today, MiniCPM-o-4_5 after the planned swap) produced a given historical judgement.
`plate_reads`	`camera_id`, `track_session_id` (FK, nullable), `plate_number`, `confidence`, `plate_image_url`, `vehicle_bbox`, `timestamp`.
`tts_cache`	`text_hash` (unique), `text`, `audio_url`, `duration_ms` — deduplicates repeated deterrent-message synthesis.
`evidence_files`	`event_id` (FK → violation_events), `file_type` (snapshot/video_clip/annotated_snapshot/metadata_json), `local_path`, `dropbox_path`, `sync_status` (pending/uploading/synced/failed), `synced_at`.
`log_entries`	Structured application log records indexed for search (feeding the Qdrant vector index described in Chapter 21): `service`, `level`, `message`, `metadata` (JSONB), `timestamp`.
`system_config`	Key-value operator-adjustable configuration (dwell thresholds, detection confidence per class, etc.), with `updated_by` / `updated_at` audit fields, so that a change to detection sensitivity is itself a traceable event.

`narration_flags` (planned — see Chapter 20)	`camera_id` (FK → cameras), `timestamp`, `flag_type` (dumping_motion/uncovered_load/anomaly), `confidence`, `narration_text`, `evidence_path`, `review_status` (pending/confirmed/dismissed), `reviewed_by`, `reviewed_at`. Deliberately kept separate from `violation_events` until a human reviewer confirms — see the governance discussion in Part I, Chapter 5.

27.2 Relationships



The relationship pattern is consistent throughout: `cameras` sits at the root of nearly every chain, since every observation the system makes is, ultimately, an observation made by a specific camera. `violation_events` is the central record most other operationally-facing tables hang off, while `track_sessions` and `vlm_analyses` form a parallel, more granular audit trail that exists independently of whether a given track ever became a full violation event — supporting after-the-fact analysis of, for example, how many tracked objects entered a zone versus how many were escalated, which is a useful operational metric distinct from the alert record itself.

27.3 API Route Groups

Route prefix	Purpose
`/api/cameras`	Camera CRUD, including ROI polygon configuration, which in turn triggers MediaMTX path provisioning and pipeline configuration reload.
`/api/roi`	Restricted-zone polygon management, decoupled from camera CRUD to support zones being edited independently of camera hardware settings.
`/api/events`	Violation event querying/filtering for the dashboard.
`/api/alerts`	Alert listing and acknowledgement (`PATCH /alerts/{id}` , restricted to admin role).
`/api/evidence`	The history browser endpoints detailed in Chapter 23.3.
`/api/lpr`	Plate read querying.
`/api/analytics`	Aggregated statistics for dashboard summary views.
`/api/system`	System configuration read/write.
`/api/devices`	Speaker/device configuration.
`/api/reports`	Scheduled/on-demand report generation.
`/api/config` , `/api/health`	Operational configuration and health-check endpoints.
`/api/narration-flags` (planned)	The review-queue API detailed in Part I, Chapter 5 and Part II, Chapter 20.

Chapter 28. Frontend and Operator Dashboard

The frontend is the surface through which every safeguard described in this document — human review, evidence inspection, acknowledgement — is actually exercised. Without a usable interface, "a human can review the evidence" is a policy statement without a mechanism. This chapter describes the operator-facing dashboard at a conceptual and user-experience level; the deep implementation details of the frontend codebase were outside the scope of this session's investigation and are not claimed here.

28.1 Live Camera View

Operators can view any configured camera's live feed via WebRTC, sourced from MediaMTX's WebRTC output (distinct from the RTSP feed DeepStream consumes for processing — see the ingestion matrix discussion in Chapter 15). The displayed stream includes the bounding-box overlays drawn by `nvosd` (Chapter 16), so an operator watching live sees exactly what the detection pipeline is currently tracking, including FSM-state-colored boxes distinguishing a merely-present object from one that has escalated to OBSERVE or VIOLATION.

28.2 TimelineView — the History Browser

The primary interface for reviewing past events is the `TimelineView` component, which calls the `GET /api/evidence/files` endpoint described in Chapter 23.3 to render a newest-first, filterable list of every captured evidence record. Each entry surfaces the keyframe snapshot, the camera and timestamp, and — where available — the AI's stated reasoning and confidence, allowing an operator to triage a day's events visually without opening each one individually, and to drill into the full clip and metadata for any event that warrants closer inspection.

28.3 Live Alerts and the Planned Review Queue

Real-time alerts arrive via the `ws_alerts` WebSocket channel and are surfaced as they occur, giving operators the option of immediate acknowledgement (`PATCH /api/alerts/{id}`) directly from the live view. Once the narration-review queue described in Chapters 5 and 20 is implemented, this is the natural surface where a "pending narration flags" list would be exposed to reviewers, following the same interaction pattern already established for alert acknowledgement — confirm or dismiss, with the identity of the reviewing operator recorded against the decision.

Chapter 29. Security, Authentication and Access Control

A system that captures and stores footage of public spaces carries a corresponding obligation to control who can access that footage and what they are permitted to do with it. This chapter documents the access-control mechanisms actually implemented in the system, distinct from the policy-level privacy commitments described in Part I, Chapter 8.

29.1 Authentication — Keycloak

All API access is authenticated against a self-hosted Keycloak realm (`keycloak_realm: "wastewatch"`), rather than a bespoke authentication implementation — a deliberate choice to rely on a mature, widely-audited open-source identity provider rather than reinventing session/token management. The API container is configured with the realm URL, client ID, and client secret needed to validate tokens issued by this realm.

29.2 Role-Based Access Control

Three roles are defined via `backend/app/auth/middleware.py`'s `require_role()` factory: `require_viewer`, `require_admin`, and `require_owner`. These are applied as FastAPI dependencies on individual routes, meaning the permission required for a given action is declared directly alongside that action's implementation rather than in a separate, easily-desynchronized configuration file.

Role	Representative permitted actions
Viewer	List and view evidence, alerts, camera feeds, plate reads — read-only across the operational surface.
Admin	Everything a viewer can do, plus acknowledging alerts (`PATCH /alerts/{id}`), and — once implemented — confirming or dismissing narration-queue entries (`POST /narration-flags/{id}/confirm` / `dismiss`).
Owner	Highest-privilege actions, expected to include system-level configuration changes and account/role management (exact route-level gating for owner-only actions was not exhaustively enumerated in this session).

> The requirement that confirming or dismissing a narration-derived flag needs admin privilege, not merely viewer access, is a deliberate governance control: the ability to promote an AI observation into an official violation record — the single action in this entire system with the most consequence for the person or vehicle involved — is restricted to a smaller set of authorized staff, not exposed to anyone with read access to the dashboard.

29.3 Path-Traversal Protection on Evidence Serving

The `GET /evidence/file` endpoint (Chapter 23.3) accepts a file path as a query parameter and serves the corresponding file directly — a pattern that, implemented carelessly, is a classic path-traversal vulnerability (a request for `../../../../etc/passwd`-style paths could otherwise read arbitrary files on the host). The implementation guards against this by resolving the requested path to its canonical absolute form and explicitly checking that it remains a descendant of `EVIDENCE_ROOT` before serving it; a request that resolves outside this boundary is rejected with an HTTP 403 rather than served.

29.4 Internal Service Communication

The gRPC channel between the pipeline and the VLM worker (Chapter 18) is configured with explicit maximum message size limits (16MB for both send and receive) — a defensive measure against a malformed or malicious oversized payload consuming unbounded memory on either side of the connection, separate from any application-level validation of the payload's contents.

Chapter 30. Deployment, Infrastructure and Capacity Planning

This chapter documents how the system is physically deployed, what hardware it requires, and how that requirement changes under the planned MiniCPM-o upgrade discussed throughout Part II.

30.1 Container Topology

The system is deployed via Docker Compose as a single-host, multi-container stack (see the architecture diagram in Chapter 13). Containers fall into three categories with materially different operational characteristics:

Category	Containers	Characteristics
GPU-bound	pipeline, vlm	Require direct GPU access, cannot be horizontally scaled without a second GPU, and represent the system's hard capacity ceiling.
Stateful data services	postgres, kafka, redis, qdrant	Require persistent volumes (`postgres_data` , `kafka_data` , `redis_data` , `qdrant_data`), standard operational backup/restore practices apply.
Stateless application services	api, alert-svc, sip-svc, evidence-svc, health-monitor, log-worker, reports, healer, lpr-worker, tts-worker	Can in principle be restarted or (with additional orchestration beyond the current Docker Compose setup) scaled horizontally without data loss, since their state lives in Postgres/Kafka rather than in-process.

30.2 GPU Capacity Budget

The current deployment targets a single NVIDIA RTX 3090 (24GB VRAM). The following table shows the current allocation and the change introduced by the planned MiniCPM-o swap:

Component	Current (Qwen2.5-VL)	Planned (MiniCPM-o 4.5)	Note
RF-DETR (TensorRT FP16)	~2 GB	~2 GB	Unchanged — detection model is not part of this upgrade.
Vision-language model	~14 GB (AWQ 4-bit)	~19 GB (BF16) or ~11 GB (int4)	MiniCPM-o 4.5 quantized (AWQ/GPTQ) checkpoints are not confirmed to exist, unlike the current Qwen2.5-VL-AWQ build — BF16 is the safer near-term assumption pending verification.
Piper TTS	~0.5 GB	~0.5 GB	CPU-bound, minimal GPU footprint regardless.
PaddleOCR (LPR)	~1 GB	~1 GB	Unchanged.
Frame buffers / overhead	~2 GB	~2-4 GB	Narration path (once live) adds concurrent session overhead proportional to active camera count — see Part II, Chapter 20's GPU-sharing discussion.
Total	**~19.5 GB / 24 GB**	**~24.5-26.5 GB / 24 GB (BF16)** or **~16.5-18.5 GB (int4)**	BF16 deployment likely exceeds available headroom on a single RTX 3090 once narration overhead is included — int4 quantization or a hardware upgrade (RTX A6000 / A100) should be treated as a probable prerequisite for the full planned scope, not an optional optimization.

> This capacity analysis is the most concrete technical risk identified anywhere in this document: the planned dual-purpose (trigger + narration) MiniCPM-o deployment, at BF16 precision, does not comfortably fit the current single-GPU budget once narration session overhead is added. Before committing to a production rollout of the narration feature specifically, GPU capacity should be re-validated against int4 quantization performance/quality trade-offs, or a hardware upgrade path should be budgeted.

30.3 Scaling Considerations

The current single-host topology is documented in the original system design specification as "Approach A: Monolithic GPU Process," selected deliberately for its operational simplicity at current camera-count scale (10-50 cameras), with two alternative approaches — a per-stage microservice architecture, and a Triton Inference Server-based orchestration layer — documented for future adoption if camera count or GPU count grows beyond what a single monolithic process can serve. Neither alternative is currently implemented; they represent forward-looking architectural options rather than available deployment modes today.

Part III — Appendices

Appendix A. Full Kafka Topic Reference

The system uses Apache Kafka as its central message bus, decoupling the real-time DeepStream pipeline from the downstream services that act on its output. Every topic below follows the naming convention `wastewatch.<domain>`. Producers publish JSON-serialized payloads (via `orjson`); consumers are grouped by Kafka consumer group so that multiple instances of the same service can share load without duplicate processing.

Topic	Producer	Consumer(s) / Group	Payload & Purpose
`wastewatch.metadata`	Pipeline (metadata extraction probe)	API metadata service (`wastewatch.group.api-metadata`)	Per-frame detection metadata (bounding boxes, classes, confidences) for live dashboard rendering and analytics aggregation.
`wastewatch.events`	Pipeline (FSM transitions)	Event-FSM consumer group; Evidence service (`wastewatch.group.evidence-recording`)	FSM state-change events (camera_id, track_id, prior state, new state, dwell_time_s, class_name, bbox, timestamp). Drives smart recording start/stop and live event timeline.
`wastewatch.vlm.requests`	Pipeline (legacy path, pre-gRPC)	VLM worker (`wastewatch.group.vlm-worker`)	Kafka-based VLM trigger request, superseded operationally by the gRPC `Analyze` RPC but retained as an alternate ingestion path with the same schema.
`wastewatch.vlm.results`	VLM worker (gRPC servicer)	Alert dispatcher	Structured VLM judgment (is_dumping, waste_type, severity, confidence, reasoning, deterrent_message, track_context) — the trigger for alert fan-out.
`wastewatch.alerts`	Alert dispatcher	WebSocket alert push, notification channels	Finalized alert payload (camera_id, track_id, severity, message) after throttle gating, ready for multi-channel delivery.
`wastewatch.lpr`	Pipeline (vehicle-in-ROI trigger)	LPR worker (`wastewatch.group.lpr-worker`)	Vehicle plate-recognition request (camera_id, track_id, vehicle_bbox, class_name).
`wastewatch.tts`	Pipeline / alert dispatcher	TTS worker	Text-to-speech generation request (camera_id, track_id, text, priority, timestamp) for on-site audio deterrent playback.
`wastewatch.evidence`	Pipeline; violation dispatcher; smart-record trigger	Evidence worker (`wastewatch.group.evidence-worker`)	Evidence file requests — snapshots and metadata JSON to be persisted and synchronized to backup storage.
`wastewatch.logs`	All services (structured logging)	Log-ingestion worker	Structured log records, ingested into Qdrant for searchable operational history.

`wastewatch.config.updates`	API (admin configuration changes)	Pipeline (hot-reload listener)	Live configuration changes (ROI polygons, class thresholds, dwell-time thresholds) applied to the running pipeline without a restart.
`wastewatch.sip`	Priority alert router; escalation logic	SIP service	Phone-call escalation triggers routed to the SIP/Twilio gateway.
`wastewatch.camera.status`	Connection monitor	Health-monitor, healer	Per-camera connectivity/health status changes.
`wastewatch.health`	Health-monitor	Healer, operations dashboard	Aggregate service and pipeline health signals.
`wastewatch.healing`	Healer	Operations dashboard, logs	Record of automated recovery actions taken (service restarts, stream reconnects).
`wastewatch.misses`	Health-monitor	Operations dashboard	Detected gaps in monitoring coverage (e.g., a camera down for a sustained period) surfaced for operator awareness.
`wastewatch.narration.flags` (planned)	Pipeline narration sampler (planned)	Narration-flag worker (planned, `wastewatch.group.narration-worker`)	Structured anomaly flags emitted by the continuous narration path (flag_type, confidence, narration_text, evidence_path) — routed into the human review queue, not directly into `violation_events`.

> Every consumer group listed above allows horizontal scaling: running multiple instances of a service under the same group ID divides the topic's partitions between them, so no message is processed twice and no service becomes a bottleneck as camera count grows.

Appendix B. Full Database Schema Reference

The system's persistent store is PostgreSQL, accessed through SQLAlchemy's async ORM with Alembic-managed migrations. The tables below represent the full schema established through migration `005` (log_entries), with the planned `narration_flags` table (migration `006`) included for completeness as a forward-looking addition.

Camera

Column	Type	Purpose
id	UUID (PK)	Primary identifier
name	String	Human-readable / pipeline-facing camera name, used to join pipeline-emitted evidence (which stores name) with DB records (which store UUID)
thumbnail_url	Text, nullable	Preview image for dashboard camera list
roi_polygons	JSONB, nullable	Operator-configured restricted-zone polygon(s) for this camera
recording_config	JSONB, nullable	Per-camera smart-recording buffer/quality settings
is_pinned	Boolean	Dashboard pin/favorite flag
last_event_at	DateTime, nullable	Timestamp of most recent event, for sort/triage
created_at / updated_at	DateTime	Audit timestamps

ViolationEvent

Column	Type	Purpose
id	UUID (PK)	Primary identifier
camera_id	UUID (FK → cameras)	Originating camera
timestamp	DateTime	Event occurrence time
event_type	Enum	Classification of the violation
severity	Enum	Alert severity level
confidence	Float	Confidence score backing this record (from FSM/VLM, or from a promoted narration flag)
track_id	Integer, nullable	Temporary tracking number active at time of event
dwell_time_s	Float, nullable	Duration the tracked object dwelled in the restricted zone
vlm_response	JSONB, nullable	Full structured AI judgment backing this record
person_id	String, nullable	Reserved field, not used for identity resolution in the current design
repeat_count	Integer	Number of prior violations attributed to the same tracked session/pattern
snapshot_url / video_clip_url	Text, nullable	Links to retained evidentiary media
status	Enum	Active / acknowledged / resolved workflow state
acknowledged_by / acknowledged_at	String / DateTime, nullable	Reviewer accountability fields

notes	Text, nullable	Free-text reviewer annotation
is_valid	Boolean, nullable	Reviewer's validity determination
created_at	DateTime	Record creation timestamp

Alert

Column	Type	Purpose
id	UUID (PK)	Primary identifier
event_id	UUID (FK → violation_events), nullable	Linked violation event, where applicable
camera_id	UUID (FK → cameras), nullable	Originating camera
severity	Enum	Alert severity
title / message	String / Text	Alert content
zone	String, nullable	Restricted zone identifier
channel	String, nullable	Delivery channel used
tts_audio_url	Text, nullable	Generated audio deterrent message, if applicable
delivered_at	DateTime, nullable	Delivery confirmation timestamp
timestamp	DateTime	Alert creation time
status	Enum	Delivery/acknowledgement workflow state
created_at	DateTime	Record creation timestamp

SystemConfig

Column	Type	Purpose
key	String (PK)	Configuration key
value	JSONB	Configuration value payload
updated_by / updated_at	String / DateTime	Change accountability

TrackSession

Column	Type	Purpose
id	UUID (PK)	Primary identifier
camera_id	UUID (FK → cameras)	Originating camera
track_id	Integer	Temporary tracker-assigned identifier
fsm_state	Enum	Current/final FSM state for this tracked session
entered_at / exited_at	DateTime	Zone entry/exit timestamps
dwell_time_s	Float, nullable	Total dwell duration
bbox_history	JSONB, nullable	Historical bounding-box path for the session
created_at	DateTime	Record creation timestamp

VLMAnalysis

Column	Type	Purpose

id	UUID (PK)	Primary identifier
track_session_id	UUID (FK → track_sessions), nullable	Linked tracking session
camera_id	UUID (FK → cameras)	Originating camera
frames_analyzed	Integer	Number of images submitted to the model for this analysis
prompt_used	Text, nullable	System prompt text sent to the model
response	JSONB	Full structured model response
inference_time_ms	Integer	Model latency for this call
model_version	String	Identifier of the model that produced this analysis (e.g. Qwen2.5-VL-7B-Instruct-AWQ, or its planned MiniCPM-o replacement)
created_at	DateTime	Record creation timestamp

AlertDelivery

Column	Type	Purpose
id	UUID (PK)	Primary identifier
alert_id	UUID (FK → alerts)	Linked alert
channel	String	Delivery channel (Telegram, SIP, speaker, WebSocket)
status	String	Delivery status (pending, sent, failed)
payload	JSONB, nullable	Channel-specific delivery payload
delivered_at	DateTime, nullable	Delivery timestamp
error_message	Text, nullable	Failure detail, if any
retry_count	Integer	Number of delivery retries attempted
created_at	DateTime	Record creation timestamp

PlateRead

Column	Type	Purpose
id	UUID (PK)	Primary identifier
camera_id	UUID (FK → cameras)	Originating camera
track_session_id	UUID (FK → track_sessions), nullable	Linked vehicle tracking session
plate_number	String	Recognized plate text
confidence	Float	OCR confidence
plate_image_url	Text, nullable	Cropped plate image
vehicle_bbox	JSONB, nullable	Vehicle bounding box at time of read
timestamp	DateTime	Read timestamp
created_at	DateTime	Record creation timestamp

TTSCacheEntry

Column	Type	Purpose

id	UUID (PK)	Primary identifier
text_hash	String, unique	Deduplication key for repeated deterrent messages
text	Text	Source text
audio_url	Text, nullable	Generated audio file location
duration_ms	Integer, nullable	Audio duration
created_at	DateTime	Record creation timestamp

EvidenceFile

Column	Type	Purpose
id	UUID (PK)	Primary identifier
event_id	UUID (FK → violation_events), nullable	Linked violation event
file_type	String	snapshot / video_clip / annotated_snapshot / metadata_json
local_path	Text, nullable	On-server file location
dropbox_path	Text, nullable	Off-site backup location
sync_status	String	pending / uploading / synced / failed
synced_at	DateTime, nullable	Backup completion timestamp
created_at	DateTime	Record creation timestamp

LogEntry

Column	Type	Purpose
id	UUID (PK)	Primary identifier
service	String	Originating service name
level	String	Log severity level
message	Text	Log message body
metadata	JSONB, nullable	Structured contextual data
timestamp	DateTime	Event time
created_at	DateTime	Ingestion timestamp

NarrationFlag (planned — migration 006)

Column	Type	Purpose
id	UUID (PK)	Primary identifier
camera_id	UUID (FK → cameras)	Originating camera
timestamp	DateTime	Observation timestamp
flag_type	String	dumping_motion / uncovered_load / anomaly
confidence	Float	Model-stated confidence for this observation
narration_text	Text	Free-text scene narration accompanying the flag
evidence_path	Text, nullable	Captured supporting clip/frame

review_status	String	pending / confirmed / dismissed
reviewed_by	String, nullable	Reviewing staff member
reviewed_at	DateTime, nullable	Review timestamp
created_at	DateTime	Record creation timestamp

> Note the structural separation: a `NarrationFlag` only becomes a `ViolationEvent` through an explicit reviewer action (see Chapter 6 and Chapter 20). The schema itself enforces the human-in-the-loop principle — there is no code path that writes directly from a narration flag into the violation table.

Appendix C. Configuration Reference

The system is configured primarily through environment variables (prefixed `WW_` for pipeline-facing settings) and a Pydantic settings object for the backend API. The table below lists every configuration value established in this document's technical review.

Variable / Setting	Component	Purpose	Default
`WW_DATABASE_URL`	Backend API	PostgreSQL connection string. Must be explicitly set — no default credentials are shipped.	(none — required)
`kafka_bootstrap_servers`	Backend API, Pipeline	Kafka broker address	localhost:9092
`redis_url`	Backend API	Redis cache connection	redis://localhost:6379/0
`mediamtx_api_url` / `mediamtx_rtsp_url`	Backend API	MediaMTX REST/RTSP endpoints for camera path management	localhost:9997 / rtsp://localhost:8554
`vlm_base_url`	Backend API (VLM client)	Base URL of the OpenAI-compatible model-serving endpoint	http://localhost:8000/v1
`vlm_model`	Backend API (VLM client)	Model identifier sent in each request; determines which model actually answers	openbmb/MiniCPM-o-4_5 (planned; Qwen/Qwen2.5-VL-7B-Instruct-AWQ live today)
`vlm_max_tokens`	Backend API (VLM client)	Maximum response length from the model	512
`vlm_temperature`	Backend API (VLM client)	Sampling temperature — kept low for deterministic, structured output	0.1
`vlm_timeout_s`	Backend API (VLM client)	HTTP request timeout for the trigger-analysis call	30
`vlm_enable_thinking`	Backend API (VLM client)	Controls whether the model's reasoning/chain-of-thought mode is enabled; disabled for the trigger path to control latency	False
`WW_VLM_GRPC_TARGET`	Pipeline	Address of the VLM worker's gRPC service for the trigger-analysis RPC	vlm-worker:50061
`WW_EVIDENCE_DIR`	Backend API, Pipeline	Root filesystem path for evidence snapshots, clips, and JSON sidecar records	/app/evidence
`telegram_bot_token` / `telegram_chat_ids`	Backend API (alerting)	Telegram notification channel credentials/targets	(none)
`ip_speaker_configs` / `ip_speaker_enabled`	Backend API	On-site audio deterrent device configuration	disabled by default
`sip_server`, `sip_username`, `sip_password`, `sip_domain`	Backend API (escalation)	SIP gateway credentials for phone-call escalation	(none)

`twilio_account_sid`, `twilio_auth_token`, `twilio_from_number`	Backend API (escalation)	Twilio-based phone escalation, alternative to raw SIP	(none)
`sip_escalation_contacts` / `sip_enabled`	Backend API	Escalation contact list and feature toggle	disabled by default
`smtp_host`, `smtp_port`, `smtp_user`, `smtp_password`, `smtp_from`, `smtp_tls`, `smtp_enabled`	Backend API (reports)	Email delivery configuration for scheduled reports	disabled by default; port 587, TLS on
`keycloak_url`, `keycloak_realm`, `keycloak_client_id`, `keycloak_client_secret`	Backend API (auth)	Identity provider configuration for staff authentication	local dev defaults; production values required per deployment
`cors_origins`	Backend API	Permitted frontend origins for the dashboard	local development origins
`WW_NARRATION_FPS` (planned)	Pipeline (narration sampler)	Sampling rate at which cached frames are submitted to the continuous narration session, per camera	to be determined during implementation/load testing (illustratively 1 fps)
`WW_NARRATION_MAX_CONCURRENT_CAMS` (planned)	Pipeline / VLM worker	Safety cap on the number of simultaneous narration sessions sharing the GPU with trigger-path calls	to be determined during load testing
`WW_NARRATION_TARGET` (planned)	Pipeline (narration client)	Address of the narration-serving endpoint (same physical service as the trigger path, different call shape)	vlm-worker:50061

Appendix D. Expanded Glossary

Term	Definition
Restricted zone	An operator-drawn polygon on a camera's field of view where dumping monitoring is active; also called a region of interest.
Region of Interest (ROI)	The technical term for a restricted zone as defined in the pipeline's spatial-filtering stage.
Dwell time	The duration a tracked object remains inside a restricted zone, measured continuously by the event FSM.
Vision-Language Model (VLM)	An AI model capable of accepting an image as input and producing natural-language interpretation or structured judgments about its content.
Confidence score	A numeric estimate, produced by the AI model itself, of how certain it is in its own judgment.
Severity	A categorical rating (e.g., info, warning, critical) assigned to a detected event, governing which notification channels are used.
Tracking ID / track_id	A temporary numeric identifier assigned to a detected object for as long as it remains visible on camera; not linked to personal identity and not persisted beyond the tracking session.
Track session	The complete lifecycle of one tracked object's presence, from entry into view/zone to exit, recorded for FSM state history.
Escalation	The process of increasing an alert's urgency or notification channel (e.g., dashboard-only → phone call) based on severity, confidence, or repetition.
Human review queue	A staged list of AI-flagged observations that require explicit staff confirmation or dismissal before being treated as an official violation record.
Finite-State Machine (FSM)	A software construct that models an object's behavior as a sequence of well-defined states (e.g., ENTER, OBSERVE, VIOLATION) with clearly specified transition conditions.
ByteTrack	The multi-object tracking algorithm used to associate detections across consecutive frames into a continuous track, using motion prediction (Kalman filtering) and detection-to-track assignment (the Hungarian algorithm).
RF-DETR	The object-detection model used to identify people and vehicles in each video frame; a transformer-based detector trained on the COCO dataset, filtered here to a small class subset relevant to dumping enforcement.
TensorRT	NVIDIA's inference-optimization runtime, used here to accelerate the detection model for real-time, low-latency execution on GPU hardware.
DeepStream	NVIDIA's GStreamer-based SDK for building GPU-accelerated video analytics pipelines; the foundation of the system's core processing pipeline.
GStreamer	The open-source multimedia framework underlying DeepStream, structuring video processing as a graph of linked elements.
Pad probe	A GStreamer mechanism allowing custom code to inspect or act on data flowing between pipeline elements without altering the pipeline's structure.
Tee	A GStreamer element that duplicates a data stream into multiple independent downstream branches.
gRPC	A high-performance remote-procedure-call framework used here to carry the trigger-analysis request from the video pipeline to the VLM worker with strict typed schemas (Protocol Buffers).
Protocol Buffers (protobuf)	The schema-definition and serialization format used by gRPC to define exact message structures such as `AnalyzeRequest`.
Circuit breaker	A reliability pattern that temporarily halts calls to a failing dependency after repeated consecutive failures, preventing cascading overload; used in the gRPC client toward the VLM worker.

Kafka topic	A named, persistent message channel in the Apache Kafka message bus; producers publish to a topic, consumers read from it.
Consumer group	A Kafka mechanism allowing multiple service instances to collectively and exclusively process a topic's messages, enabling horizontal scaling without duplicate processing.
Fire-and-forget	A call pattern where the caller submits a request and continues immediately without waiting for or blocking on the response, used for the video pipeline's trigger call so real-time frame processing is never delayed.
Omni-Flow	MiniCPM-o's internal architecture for real-time multimodal streaming, aligning video (and optionally audio) processing to a shared time-chunked timeline rather than discrete request/response turns.
Time-Aligned Interleaving (TAIL)	The mechanism within Omni-Flow that keeps generated output synchronized with the real-time input timeline, preventing the model's responses from drifting behind the live video feed.
vllm-omni	A serving framework, extending the open-source vLLM inference engine, capable of hosting omni-modal models like MiniCPM-o and exposing both standard chat-completion and native streaming interfaces.
Duplex streaming session	A persistent, stateful connection to a model that accumulates context across many sequential inputs, as opposed to a stateless one-shot request; used for the planned continuous narration path.
`streaming_prefill` / `streaming_generate`	The two calls composing a duplex streaming interaction — feeding a new chunk of input into the session's running context, and pulling generated output for that chunk, respectively.
Chat completion / chat_template_kwargs	The standard request format for instructing a language model via a structured message list; `chat_template_kwargs` carries model-specific behavioral flags, such as disabling reasoning mode.
Reasoning / "thinking" mode	A model behavior in which additional intermediate reasoning tokens are generated before a final answer, improving accuracy at the cost of latency; disabled for the system's latency-sensitive trigger path.
JSONB	A PostgreSQL column type storing structured JSON data in a queryable binary format, used throughout the schema for flexible AI-response and configuration storage.
Alembic migration	A versioned, reproducible script describing a single change to the database schema, allowing the schema to evolve safely across deployments.
Object-Relational Mapper (ORM)	A software layer (SQLAlchemy, in this system) that lets application code interact with database tables as Python objects rather than raw SQL.
Smart recording	The evidence subsystem's behavior of automatically starting a video recording buffer slightly before a violation begins and continuing slightly after it ends, capturing full context rather than a single frame.
Chain of custody	The documented, auditable trail proving that a piece of evidence has been handled and stored in a controlled, unaltered manner since its creation.
License Plate Recognition (LPR)	The automated process of detecting and reading a vehicle's license plate from a camera image, used here for vehicles observed in restricted zones.
PaddleOCR	The optical-character-recognition engine used to read license plate text from cropped vehicle images.
Text-to-Speech (TTS)	The automated conversion of text (such as a deterrent message) into spoken audio for on-site playback.
Piper	The lightweight, CPU-efficient text-to-speech engine used to generate on-site audio announcements.
Role-based access control (RBAC)	A permissions model in which a user's allowed actions are determined by their assigned role (e.g., viewer, administrator, owner) rather than being individually configured per user.

Keycloak	The identity and access-management system used to authenticate staff and issue role-scoped credentials.
WebSocket	A persistent, bidirectional network connection protocol used to push live metadata, events, and alerts to the operator dashboard in real time.
WebRTC	A real-time media-streaming protocol used to deliver low-latency live camera video to the operator dashboard's browser-based viewer.
MediaMTX	The media-server component that ingests camera streams in whatever protocol they arrive (RTSP/RTMP/HTTPS) and normalizes them for both pipeline processing and dashboard viewing.
Docker Compose	The container-orchestration tool used to define and run the full multi-service system as a coordinated set of Docker containers.
VRAM (video memory)	The dedicated memory on a GPU, a critical sizing constraint for how many AI models (detector, tracker overhead, VLM) can run concurrently on a given server.
Quantization (e.g., AWQ, int4, BF16)	Techniques for reducing a model's numeric precision to lower its memory footprint and increase inference speed, at a generally small accuracy cost.

Appendix E. Diagram Index

This document is deliberately diagram-heavy: nearly every chapter in Parts I and II includes at least one flowchart illustrating the process, state machine, or architecture it describes, rather than relying on prose alone. Each of the following chapters includes an accompanying flowchart:

1. Chapter 3 — System Overview and Plain-Language Flow
2. Chapter 4 — Detection Stage in Detail
3. Chapter 5 — Verification Stage — AI Visual Analysis
4. Chapter 6 — Human Oversight and Review Governance
5. Chapter 7 — Alerting and Escalation Process
6. Chapter 8 — Evidence Handling and Chain of Custody
7. Chapter 10 — Reliability, Monitoring and Failure Handling
8. Chapter 11 — Roadmap — Planned Enhancements
9. Chapter 13 — System Architecture Overview
10. Chapter 14 — Container Inventory
11. Chapter 16 — DeepStream Pipeline Internals
12. Chapter 17 — Event FSM Deep Dive
13. Chapter 18 — VLM Trigger Path Deep Dive
14. Chapter 19 — MiniCPM-o 4.5 Model Internals and Omni-Flow
15. Chapter 20 — Planned Narration Path Deep Dive
16. Chapter 21 — Kafka Messaging Layer
17. Chapter 22 — Alert and Escalation Services
18. Chapter 23 — Evidence and Recording Subsystem
19. Chapter 26 — Health Monitoring and Self-Healing
20. Chapter 27 — Backend API and Database Schema
21. Chapter 30 — Deployment, Infrastructure and Capacity Planning

Diagram numbering and exact figure counts are finalized in the chapters themselves; this index reflects the chapters carrying primary flowcharts as authored across Parts I and II.

Appendix F. Document Change Log and Versioning

Version	Date	Change Summary
1.0	August 2026	Initial comprehensive edition — combined governance narrative and full technical architecture reference, including the planned MiniCPM-o 4.5 trigger-model upgrade and continuous narration enhancement.

Subsequent versions of this document should record, at minimum: the date the planned narration enhancement moves from specified/planned to deployed status; any change to human-review workflow; and any change to data retention or privacy-relevant handling.

Appendix G. Testing and Validation Summary

The MiniCPM-o 4.5 trigger-model swap and continuous-narration enhancement were developed against a test-driven implementation plan, summarized here to give a reviewer visibility into how the planned changes are being validated before any production deployment.

Phase 1 — Trigger model swap validation

- A dedicated settings test confirms the VLM configuration defaults correctly point at the new model identifier and that the reasoning-mode flag defaults to disabled.
- A VLM client test confirms that the outbound request payload actually carries the `chat_template_kwargs` reasoning-mode flag, using a mocked HTTP layer rather than a live model call — verifying the client's behavior without requiring GPU hardware or a deployed model server.
- Existing VLM client tests (successful analysis, parse-failure retry, double-failure fallback, HTTP-error fallback) are re-run to confirm no regression in the retry/fallback behavior that keeps the pipeline resilient to malformed or absent model responses.

Phase 2 — Narration path validation

- The narration client is implemented behind a fixed contract (`submit_frame() → {narration, flags}`) with a stub backend for initial testing, allowing the surrounding pipeline wiring, Kafka publishing, database persistence, and review API to be fully validated without a live GPU deployment of the real streaming model.
- A dedicated Kafka producer test confirms the new narration-flag topic is published to correctly.
- A database model test and a migration confirm the new review-queue table's schema matches specification before any data is written to it.
- A worker test confirms that a simulated flag message is correctly resolved to the originating camera and persisted with a pending review status.
- An API test confirms that a reviewer's confirmation action correctly transitions a flag's status, records the reviewer's identity, and creates a corresponding official violation record — directly testing the human-oversight safeguard described in Chapter 6.
- An end-to-end simulator test chains a synthetic flag message through the worker, into the database, and out through the listing API, confirming the full pipeline without requiring a live camera feed or a deployed narration model.

> The real vllm-omni deployment and the swap of the narration client's stub backend for an actual duplex-streaming call to MiniCPM-o are explicitly deferred to a follow-up validation phase, to be conducted once GPU capacity is confirmed and the serving framework's production maturity is assessed under real load — this is disclosed here rather than presented as already complete.

Appendix H. Compliance and Governance Principles Mapping

The table below maps specific design choices documented in this report to widely recognized data-protection and algorithmic-oversight principles. It is presented generically, without reference to any specific jurisdiction's statute, since the applicable legal framework depends on where the system is deployed; a legal reviewer should map these design choices against the specific regulations governing the deployment jurisdiction.

Principle	System Design Choice
Purpose limitation	Monitoring and evaluation is confined to operator-defined restricted zones tied to a specific enforcement purpose (dumping prevention); activity elsewhere in a camera's field of view is not evaluated or retained (Chapter 4).
Data minimization	No facial recognition is performed; detected individuals are represented only by a temporary, non-identifying tracking number discarded once they leave camera view (Chapter 9).
Human oversight of automated decisions	No automated step independently produces a final violation record; the structured verification path requires FSM-plus-AI agreement, and the open-ended narration path requires explicit human confirmation before any record is created (Chapters 5, 6, 20).
Accountability and auditability	Every review decision is logged with the reviewing staff member's identity and timestamp; every evidentiary record retains its full supporting context (Chapters 6, 8).
Security of processing	AI inference is performed entirely on infrastructure controlled by the operating authority; no imagery is transmitted to third-party commercial AI services (Chapter 5). Evidence access is gated by authenticated, role-scoped credentials (Chapter 29).
Transparency	This document itself — describing, at both governance and technical depth, exactly how detection, verification, and escalation decisions are made — constitutes the system's transparency mechanism toward regulatory and oversight bodies.
Storage limitation and integrity	Evidentiary records are timestamped and treated as immutable once written, supporting a defensible retention and audit posture; retention-period policy is an operational configuration decision left to the deploying authority (Chapter 8).
Proportionality of escalation	Notification and escalation channel selection (dashboard vs. messaging vs. phone call) scales with confirmed severity and repetition, rather than treating every flagged event identically (Chapter 7).